Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 04, Companion Volume. First Edition, 8 November 2026. This guide contains the practical assignments, 120+ MCQs, short-answer questions, scenario-based troubleshooting questions, incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your VM. That is where the learning happens.

Assessment and Answer Guide — মূল্যায়ন ও উত্তর নির্দেশিকা

এই অধ্যায়ে বইয়ের ২৪টি অধ্যায়ের ওপর হাতে-কলমে মূল্যায়ন দেওয়া হয়েছে। প্রথমে চেষ্টা করুন, তারপর শেষের Answer Key দেখুন — উত্তর আগে দেখলে শেখা কমে। সব উদাহরণে ফিকশনাল ডোমেইন technova.example ও TEST-NET IP 203.0.113.x ব্যবহৃত হয়েছে; আসল সার্ভারে চালানোর আগে মান বদলে নিন।

সারাংশ — এই অধ্যায়ে যা আছে

অংশ	সংখ্যা	বিবরণ
ব্যবহারিক অ্যাসাইনমেন্ট	২০টি	প্রতিটিতে কাজ, ডেলিভারেবল ও মূল্যায়ন-ইঙ্গিত
বহুনির্বাচনি প্রশ্ন (MCQ)	১২০টি	MCQ ১–১২০, প্রতিটিতে ৪টি বিকল্প (A–D), ২৪ অধ্যায় জুড়ে
সংক্ষিপ্ত উত্তর (SA)	৩০টি	SA ১–৩০
পরিস্থিতিভিত্তিক সমস্যা (SC)	২০টি	SC ১–২০, বাস্তব প্রোডাকশন পরিস্থিতি
ইনসিডেন্ট কেস স্টাডি (CS)	১০টি	CS ১–১০, ঘটনা-বিবরণ + বিশ্লেষণ প্রশ্ন
চূড়ান্ত ব্যবহারিক পরীক্ষা	১টি	তত্ত্ব + হাতে-কলমে, নম্বর ও সময়সহ
ক্যাপস্টোন রুব্রিক	১টি	পরিমাপযোগ্য মানদণ্ডের টেবিল
উত্তর নির্দেশিকা (Answer Key)	—	সব MCQ-র ব্যাখ্যাসহ উত্তর, SA/SC-র মডেল উত্তর, CS ও পরীক্ষার মূল্যায়ন-নোট

> **সতর্কতা:** অ্যাসাইনমেন্টের ধ্বংসাত্মক ধাপ (ফাইল মোছা, ফায়ারওয়াল বদল, ডিস্ক ফরম্যাট) শুধু ল্যাব VM-এ করবেন। প্রোডাকশনে করার আগে ব্যাকআপ নিন ও ফেরার পথ নিশ্চিত করুন।

ব্যবহারিক অ্যাসাইনমেন্ট (২০টি)

প্রতিটি অ্যাসাইনমেন্ট ল্যাব VM (Ubuntu Server 24.04 LTS)-এ করবেন। ডেলিভারেবল জমা দেওয়ার সময় কমান্ডের আউটপুট নমুনা আউটপুট — চিত্রণমূলক হিসেবে চিহ্নিত করবেন।

1. **অ্যাসাইনমেন্ট 1: প্রোডাকশন সার্ভার পরিকল্পনা ও Ubuntu ইনস্টল**

   কাজ: technova.example-এর জন্য একটি প্রোডাকশন সার্ভার পরিকল্পনা করুন: Ubuntu Server 24.04 LTS ইনস্টল, পার্টিশন স্কিম (/, /var, /home, swap), hostname, timezone Asia/Dhaka, এবং প্রথম বুট চেকলিস্ট।

   ডেলিভারেবল: পরিকল্পনা নথি (২–৩ পৃষ্ঠা), পার্টিশন টেবিল, ইনস্টল-পরবর্তী যাচাই কমান্ডের আউটপুট (নমুনা আউটপুট — চিত্রণমূলক)।

   মূল্যায়ন-ইঙ্গিত: পার্টিশন যুক্তি সঠিক কি না, /var আলাদা করার কারণ ব্যাখ্যা হয়েছে কি না, যাচাই ধাপ সম্পূর্ণ কি না।

2. **অ্যাসাইনমেন্ট 2: SSH হার্ডেনিং ও ব্যবহারকারী ব্যবস্থাপনা**

   কাজ: একটি সার্ভারে root SSH লগইন বন্ধ, key-based login চালু, sudo ব্যবহারকারী তৈরি, fail2ban-সদৃশ সুরক্ষা কনফিগার করুন (ল্যাব VM-এ)।

   ডেলিভারেবল: sshd_config পরিবর্তনের তালিকা, ব্যবহারকারী ও sudo নীতি, লগইন পরীক্ষার লগ।

   মূল্যায়ন-ইঙ্গিত: key ছাড়া লগইন ব্যর্থ হয় কি না, root সরাসরি লগইন বন্ধ কি না, sudo লগ যাচাই করা হয়েছে কি না।

3. **অ্যাসাইনমেন্ট 3: ফাইলসিস্টেম ও LVM**

   কাজ: LVM দিয়ে /data ভলিউম তৈরি করুন, পরে তা ৫ GB বাড়ান, snapshot নিয়ে ফেরত (restore) পরীক্ষা করুন।

   ডেলিভারেবল: কমান্ড ইতিহাস, lsblk/df আউটপুট, snapshot পরীক্ষার প্রতিবেদন।

   মূল্যায়ন-ইঙ্গিত: resize-এর আগে-পরে আকার, snapshot থেকে ফেরতের প্রমাণ, ডেটা অখণ্ডতা যাচাই।

4. **অ্যাসাইনমেন্ট 4: প্রসেস ও রিসোর্স পর্যবেক্ষণ**

   কাজ: একটি CPU-ক্ষুধার্ত প্রসেস শনাক্ত করে nice/renice ও cgroup-সদৃশ সীমা প্রয়োগ করুন; top/htop ও ps আউটপুট বিশ্লেষণ করুন।

   ডেলিভারেবল: পর্যবেক্ষণ প্রতিবেদন, শনাক্তকরণের যুক্তি, প্রয়োগ করা সীমা।

   মূল্যায়ন-ইঙ্গিত: সঠিক প্রসেস শনাক্ত, সীমার প্রভাব পরিমাপ, ভুল প্রসেস kill না করা।

5. **অ্যাসাইনমেন্ট 5: প্যাকেজ ব্যবস্থাপনা ও আপডেট নীতি**

   কাজ: apt দিয়ে নিরাপত্তা আপডেটের সময়সূচি, unattended-upgrades কনফিগার, একটি প্যাকেজের নির্দিষ্ট সংস্করণ hold/unhold করুন।

   ডেলিভারেবল: আপডেট নীতি নথি, কনফিগ ফাইল, পরীক্ষার লগ।

   মূল্যায়ন-ইঙ্গিত: reboot-প্রয়োজনীয় আপডেট চিহ্নিত কি না, hold যুক্তি স্পষ্ট কি না।

6. **অ্যাসাইনমেন্ট 6: systemd দিয়ে নিজের সার্ভিস**

   কাজ: একটি Node.js অ্যাপের জন্য systemd unit লিখুন: restart policy, EnvironmentFile, WorkingDirectory, journald লগ।

   ডেলিভারেবল: app.service ফাইল, systemctl status আউটপুট, লগ নমুনা।

   মূল্যায়ন-ইঙ্গিত: restart আচরণ, পরিবেশ ভেরিয়েবল লোড, লগে ত্রুটি দৃশ্যমান কি না।

7. **অ্যাসাইনমেন্ট 7: Nginx ভার্চুয়াল হোস্ট ও রিভার্স প্রক্সি**

   কাজ: দুটি ডোমেইন (app.technova.example, api.technova.example) — একটি static, একটি Node রিভার্স প্রক্সি — কনফিগার করুন।

   ডেলিভারেবল: দুটি server block, nginx -t আউটপুট, curl পরীক্ষা।

   মূল্যায়ন-ইঙ্গিত: Host হেডার রাউটিং, proxy header সঠিক কি না, static ও API আলাদা কি না।

8. **অ্যাসাইনমেন্ট 8: TLS ও Let's Encrypt**

   কাজ: Certbot দিয়ে TLS চালু করুন, HTTP→HTTPS redirect, HSTS ও স্বয়ংক্রিয় নবায়ন পরীক্ষা করুন।

   ডেলিভারেবল: সার্টিফিকেট বিবরণ, redirect পরীক্ষা, নবায়ন dry-run লগ।

   মূল্যায়ন-ইঙ্গিত: নবায়ন স্বয়ংক্রিয় কি না, mixed content নেই কি না, redirect চেইন সঠিক কি না।

9. **অ্যাসাইনমেন্ট 9: Node.js প্রোডাকশন ডেপ্লয়মেন্ট**

   কাজ: একটি Express + MongoDB অ্যাপ production build, NODE_ENV=production, reverse proxy-র পেছনে চালান।

   ডেলিভারেবল: ডেপ্লয় ধাপের নথি, PM2 বা systemd পছন্দের যুক্তি, health endpoint পরীক্ষা।

   মূল্যায়ন-ইঙ্গিত: production মোড যাচাই, পোর্ট সরাসরি উন্মুক্ত নয় কি না, health check কাজ করে কি না।

10. **অ্যাসাইনমেন্ট 10: PM2 বনাম systemd তুলনা**

    কাজ: একই অ্যাপ PM2 ও systemd — দুভাবেই চালিয়ে তুলনা করুন: restart, লগ, cluster, boot-এ চালু।

    ডেলিভারেবল: তুলনা টেবিল, দুটি কনফিগ, সিদ্ধান্ত ও যুক্তি।

    মূল্যায়ন-ইঙ্গিত: তুলনা তথ্যভিত্তিক কি না, কখন কোনটি — সিদ্ধান্ত স্পষ্ট কি না।

11. **অ্যাসাইনমেন্ট 11: PostgreSQL ও MongoDB পরিচালনা**

    কাজ: PostgreSQL-এ ব্যবহারকারী/ডেটাবেস/ব্যাকআপ (pg_dump) এবং MongoDB-তে auth ও mongodump — দুটিই করুন।

    ডেলিভারেবল: ব্যাকআপ ফাইল তালিকা, restore পরীক্ষা, auth পরীক্ষা।

    মূল্যায়ন-ইঙ্গিত: restore সত্যিই কাজ করে কি না, auth ছাড়া প্রবেশ বন্ধ কি না।

12. **অ্যাসাইনমেন্ট 12: ডিস্ক ও inode সংকট সমাধান**

    কাজ: ইচ্ছাকৃতভাবে /var/log ভরিয়ে (ল্যাব) ডিস্ক পূর্ণের কারণ খুঁজুন, logrotate ঠিক করুন, inode ব্যবহার পরীক্ষা করুন।

    ডেলিভারেবল: কারণ বিশ্লেষণ, সমাধান ধাপ, logrotate কনফিগ।

    মূল্যায়ন-ইঙ্গিত: লক্ষণ নয়, মূল কারণ ধরা হয়েছে কি না; মুছে ফেলার আগে যাচাই।

13. **অ্যাসাইনমেন্ট 13: নেটওয়ার্কিং ও ফায়ারওয়াল**

    কাজ: UFW দিয়ে শুধু 22/80/443 খুলুন, nftables নিয়মের সঙ্গে তুলনা করুন, ss দিয়ে listening পোর্ট যাচাই করুন।

    ডেলিভারেবল: ফায়ারওয়াল নিয়ম তালিকা, পোর্ট স্ক্যান ফল, তুলনা নোট।

    মূল্যায়ন-ইঙ্গিত: অপ্রয়োজনীয় পোর্ট বন্ধ কি না, নিজেকে lockout না করা।

14. **অ্যাসাইনমেন্ট 14: সার্ভার হার্ডেনিং অডিট**

    কাজ: একটি সার্ভারের অডিট করুন: অপ্রয়োজনীয় সার্ভিস, বিশ্ব-লেখযোগ্য ফাইল, SUID, SSH, আপডেট অবস্থা — প্রতিবেদন দিন।

    ডেলিভারেবল: অডিট প্রতিবেদন ও সংশোধন তালিকা, অগ্রাধিকার ক্রম।

    মূল্যায়ন-ইঙ্গিত: ঝুঁকি অগ্রাধিকার সঠিক কি না, সংশোধন যাচাই হয়েছে কি না।

15. **অ্যাসাইনমেন্ট 15: ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা**

    কাজ: rsync + cron দিয়ে দৈনিক ব্যাকআপ, 3-2-1 নীতি, এবং একটি ফাইল মুছে restore করে প্রমাণ করুন।

    ডেলিভারেবল: ব্যাকআপ স্ক্রিপ্ট, cron entry, restore পরীক্ষার লগ।

    মূল্যায়ন-ইঙ্গিত: restore পরীক্ষা ছাড়া ব্যাকআপ সম্পূর্ণ ধরা হয়নি কি না।

16. **অ্যাসাইনমেন্ট 16: লগ ও journald বিশ্লেষণ**

    কাজ: journald ও Nginx access/error লগ থেকে একটি ব্যর্থ ডেপ্লয়ের কারণ খুঁজুন; logrotate ও retention নির্ধারণ করুন।

    ডেলিভারেবল: ঘটনা-সময়রেখা, মূল কারণ, retention নীতি।

    মূল্যায়ন-ইঙ্গিত: সময়রেখা লগ-প্রমাণভিত্তিক কি না, অনুমান আলাদা করা হয়েছে কি না।

17. **অ্যাসাইনমেন্ট 17: MERN স্ট্যাক সম্পূর্ণ ডেপ্লয়**

    কাজ: MongoDB + Express + React (build) + Node — একটি MERN অ্যাপ সার্ভারে ডেপ্লয় করুন, env ভেরিয়েবল .env ফাইলে (ফিকশনাল মান)।

    ডেলিভারেবল: ডেপ্লয় রানবুক, আর্কিটেকচার চিত্র, smoke test তালিকা।

    মূল্যায়ন-ইঙ্গিত: প্রতিটি স্তর আলাদাভাবে যাচাই, secret git-এ যায়নি কি না।

18. **অ্যাসাইনমেন্ট 18: ডেপ্লয় কৌশল ও রোলব্যাক**

    কাজ: Blue-green বা rolling ডেপ্লয়ের পরিকল্পনা করুন, ব্যর্থ হলে ৫ মিনিটে রোলব্যাকের মহড়া দিন।

ডেলিভারেবল: ডেপ্লয় পরিকল্পনা, রোলব্যাক ধাপ, মহড়ার সময় পরিমাপ।

মূল্যায়ন-ইঙ্গিত: রোলব্যাক ধাপ পরীক্ষিত কি না, ডেটা মাইগ্রেশন ঝুঁকি বিবেচিত কি না।

19. **অ্যাসাইনমেন্ট 19: ইনসিডেন্ট রেসপন্স মহড়া**

    কাজ: 'ওয়েবসাইট 502' ইনসিডেন্টের মহড়া: triage, যোগাযোগ, সমাধান, পোস্টমর্টেম লিখুন।

    ডেলিভারেবল: ইনসিডেন্ট টাইমলাইন, পোস্টমর্টেম, প্রতিরোধমূলক কাজের তালিকা।

    মূল্যায়ন-ইঙ্গিত: দোষারোপ নয়, মূল কারণ ও প্রতিরোধ — স্পষ্ট কি না।

20. **অ্যাসাইনমেন্ট 20: ক্যাপস্টোন: প্রোডাকশন সার্ভার**

    কাজ: নিজের MERN প্রজেক্ট একটি Ubuntu সার্ভারে সম্পূর্ণ প্রোডাকশনে নিন: TLS, ব্যাকআপ, মনিটরিং, হার্ডেনিং, রানবুক — সবসহ।

    ডেলিভারেবল: লাইভ URL (ল্যাব), রানবুক, চেকলিস্ট, পোস্টমর্টেম টেমপ্লেট।

    মূল্যায়ন-ইঙ্গিত: রুব্রিকের সব মানদণ্ড পূরণ, প্রতিটি দাবির প্রমাণ (লগ/স্ক্রিনশট)।

বহুনির্বাচনি প্রশ্ন (MCQ ১–১২০)

প্রতিটি প্রশ্নে একটি সঠিক উত্তর আছে। এই প্রশ্নগুলো শুধু কমান্ডের নাম মুখস্থ পরীক্ষা করে না — বোঝাপড়া ও সিদ্ধান্ত যাচাই করে। উত্তর শেষের Answer Key-তে আছে; আগে নিজে চেষ্টা করুন।

1. **MCQ 1.** প্রোডাকশন সার্ভার আর ডেভেলপমেন্ট ল্যাপটপের সবচেয়ে বড় পার্থক্য কোনটি?
    1. সার্ভারে বেশি RAM থাকে
    2. সার্ভার ২৪/৭ চলতে হয়, ব্যর্থতার খরচ সরাসরি ব্যবহারকারীকে আঘাত করে
    3. সার্ভারে GUI থাকে না
    4. সার্ভার দামি
2. **MCQ 2.** Ubuntu Server 24.04 LTS বেছে নেওয়ার সবচেয়ে যুক্তিসঙ্গত কারণ কী?
    1. এটি সবচেয়ে নতুন সংস্করণ
    2. দীর্ঘমেয়াদি নিরাপত্তা আপডেট ও স্থিতিশীলতা পাওয়া যায়
    3. এতে সব প্যাকেজের সর্বশেষ সংস্করণ থাকে
    4. ইনস্টল দ্রুত হয়
3. **MCQ 3.** /var আলাদা পার্টিশনে রাখার প্রধান সুবিধা কী?
    1. সার্ভার দ্রুত বুট হয়
    2. লগ বা ডেটা বেড়ে গেলে root ফাইলসিস্টেম ভরে সিস্টেম থেমে যাওয়া ঠেকায়
    3. ডিস্কের আয়ু বাড়ে
    4. ব্যাকআপ ছোট হয়
4. **MCQ 4.** সার্ভার ইনস্টলের সময় swap নিয়ে সঠিক সিদ্ধান্ত কোনটি?
    1. RAM যতই থাকুক swap লাগবেই
    2. swap না থাকলে সার্ভার চলবে না
    3. RAM ও কাজের ধরন দেখে সিদ্ধান্ত নিতে হয়; swap নেই মানেই ভুল নয়
    4. swap যত বড় তত ভালো
5. **MCQ 5.** প্রোডাকশনে প্রথম বুটের পর প্রথম কাজ কোনটি?
    1. অ্যাপ ডেপ্লয় করা
    2. সিস্টেম আপডেট ও নিরাপত্তা প্যাচ যাচাই করা
    3. Nginx ইনস্টল করা
    4. ডোমেইন কেনা
6. **MCQ 6.** SSH key-based লগইন পাসওয়ার্ড লগইনের চেয়ে নিরাপদ কেন?
    1. key মনে রাখা সহজ
    2. key brute-force করা কার্যত অসম্ভব দীর্ঘ ও সার্ভারে পাসওয়ার্ড অনুমান আক্রমণ কাজ করে না
    3. key কখনো চুরি হয় না

    4. key ছোট হয়

7. **MCQ 7.** root দিয়ে সরাসরি SSH লগইন বন্ধ করা উচিত কেন?
    1. root ধীরগতির
    2. root লগইনে কে কী করল তা শনাক্ত করা যায় না এবং আক্রমণের লক্ষ্য একটাই হয়
    3. root-এর পাসওয়ার্ড ভুলে যাওয়া যায়
    4. sudo দ্রুততর

8. **MCQ 8.** একজন ডেভেলপারকে সার্ভারে সীমিত অধিকার দিতে সবচেয়ে ভালো উপায় কী?
    1. root পাসওয়ার্ড শেয়ার করা
    2. sudo-তে শুধু প্রয়োজনীয় কমান্ডের অনুমতি দেওয়া
    3. সবাইকে একই অ্যাকাউন্ট দেওয়া
    4. পাসওয়ার্ড না দেওয়া

9. **MCQ 9.** sudo লগ রাখা জরুরি কেন?
    1. ডিস্ক ভরতে
    2. কে কখন প্রশাসনিক কাজ করল তার হিসাব রাখতে
    3. সার্ভার দ্রুত করতে
    4. ব্যাকআপ নিতে

10. **MCQ 10.** SSH পোর্ট 22 থেকে অন্য পোর্টে সরানোকে কীভাবে দেখা উচিত?
    1. এটাই সম্পূর্ণ সুরক্ষা
    2. এটি শুধু শব্দ কমায়; key, ফায়ারওয়াল ও fail2ban-ই আসল সুরক্ষা
    3. এতে লগইন অসম্ভব হয়
    4. এটি বাধ্যতামূলক

11. **MCQ 11.** LVM ব্যবহারের প্রধান সুবিধা কী?
    1. ডিস্ক দ্রুত হয়
    2. পার্টিশনের আকার পরে বাড়ানো-কমানো যায়
    3. ডেটা স্বয়ংক্রিয় ব্যাকআপ হয়
    4. ফাইলসিস্টেম বদলে যায়

12. **MCQ 12.** ফাইলসিস্টেম পূর্ণ হওয়ার আগে সতর্ক হওয়ার সবচেয়ে ভালো উপায় কী?
    1. প্রতিদিন হাতে df চালানো
    2. ডিস্ক ব্যবহার মনিটরিং ও সতর্কতা (alert) বসানো
    3. বড় ফাইল মুছে ফেলা
    4. reboot করা

13. **MCQ 13.** একটি ফাইল মুছে ফেলার পরও df-তে জায়গা না কমলে সম্ভাব্য কারণ কী?
    1. ফাইলটি আসলে মুছে যায়নি

2. কোনো চলমান প্রসেস ফাইলটি খুলে রেখেছে
3. ডিস্ক নষ্ট
4. reboot দরকার

14. **MCQ 14.** inode শেষ হয়ে গেলে কী হয়?
    1. ডিস্কের জায়গা শেষ হয়
    2. জায়গা থাকলেও নতুন ফাইল তৈরি করা যায় না
    3. সার্ভার reboot হয়
    4. RAM শেষ হয়

15. **MCQ 15.** অনেক ছোট ফাইল তৈরি করা অ্যাপের জন্য কোন সতর্কতা সবচেয়ে জরুরি?
    1. ফাইলের নাম ছোট রাখা
    2. inode ব্যবহার পর্যবেক্ষণ করা
    3. ফাইল compress করা
    4. RAM বাড়ানো

16. **MCQ 16.** chmod 777 দেওয়া কেন বিপজ্জনক?
    1. ফাইল ধীর হয়
    2. যে কেউ ফাইল বদলাতে বা চালাতে পারে
    3. ফাইল মুছে যায়
    4. ব্যাকআপ বন্ধ হয়

17. **MCQ 17.** একটি কনফিগ ফাইলের মালিকানা root:root ও অনুমতি 600 হওয়া যুক্তিসঙ্গত কখন?
    1. সব ফাইলের জন্য
    2. গোপন তথ্য (secret/key) থাকা ফাইলের জন্য
    3. public HTML-এর জন্য
    4. লগ ফাইলের জন্য

18. **MCQ 18.** হার্ড লিংক ও সিমলিংকের পার্থক্য বোঝা জরুরি কেন?
    1. হার্ড লিংক দ্রুত
    2. সিমলিংক ভাঙলে মূল ফাইল মুছে যায় না, কিন্তু হার্ড লিংক মুছলে inode থাকে
    3. ব্যাকআপ ও মুছে ফেলার আচরণ আলাদা হয়
    4. সিমলিংক বেশি জায়গা নেয়

19. **MCQ 19.** সার্ভারে /tmp-এ অস্থায়ী ফাইল জমে গেলে সঠিক প্রতিক্রিয়া কী?
    1. সব মুছে ফেলা
    2. কোন প্রসেস/অ্যাপ তৈরি করছে তা খুঁজে মূল কারণ ঠিক করা
    3. /tmp বড় করা
    4. reboot করা

20. **MCQ 20.** ফাইলসিস্টেম চেক (fsck) কখন চালানো উচিত?
    1. প্রতিদিন
    2. সন্দেহজনক ত্রুটি বা unclean shutdown-এর পর, সতর্কভাবে
    3. কখনোই নয়
    4. শুধু ডিস্ক পূর্ণ হলে

21. **MCQ 21.** load average 8.0 (4 CPU) দেখে কী বোঝা যায়?
    1. সার্ভার স্বাভাবিক
    2. CPU-র চেয়ে বেশি কাজ অপেক্ষায় — চাপ আছে
    3. RAM শেষ
    4. ডিস্ক শেষ

22. **MCQ 22.** একটি প্রসেস CPU 100% নিচ্ছে — প্রথমে কী করা উচিত?
    1. সঙ্গে সঙ্গে kill করা
    2. প্রসেসটি কী, কার, কী করছে — শনাক্ত করা
    3. reboot করা
    4. RAM বাড়ানো

23. **MCQ 23.** zombie প্রসেস কী নির্দেশ করে?
    1. RAM শেষ
    2. প্যারেন্ট প্রসেস সন্তানের সমাপ্তি সংগ্রহ করেনি
    3. ডিস্ক পূর্ণ
    4. নেটওয়ার্ক বন্ধ

24. **MCQ 24.** SIGTERM ও SIGKILL-এর মধ্যে সঠিক পার্থক্য কোনটি?
    1. দুটোই এক
    2. SIGTERM সুযোগ দেয় পরিষ্কারভাবে থামতে, SIGKILL জোর করে থামায়
    3. SIGKILL ধীর
    4. SIGTERM শুধু root-এর জন্য

25. **MCQ 25.** nice মান বাড়ালে কী হয়?
    1. প্রসেস দ্রুত হয়
    2. প্রসেসের CPU অগ্রাধিকার কমে
    3. RAM বাড়ে
    4. প্রসেস থেমে যায়

26. **MCQ 26.** apt update ও apt upgrade-এর পার্থক্য কী?
    1. দুটোই এক
    2. update তালিকা হালনাগাদ করে, upgrade প্যাকেজ ইনস্টল করে

3. update প্যাকেজ মুছে ফেলে
4. upgrade তালিকা হালনাগাদ করে

27. **MCQ 27.** প্রোডাকশনে unattended-upgrades চালুর ঝুঁকি কী?
    1. আপডেট হয় না
    2. স্বয়ংক্রিয় আপডেটে কখনো সার্ভিস restart বা আচরণ বদলাতে পারে
    3. ডিস্ক ভরে যায়
    4. SSH বন্ধ হয়

28. **MCQ 28.** একটি প্যাকেজের নির্দিষ্ট সংস্করণ আটকে (hold) রাখা কখন যুক্তিসঙ্গত?
    1. সবসময়
    2. অ্যাপটি ওই সংস্করণের ওপর নির্ভরশীল ও নতুন সংস্করণ পরীক্ষিত নয়
    3. ডিস্ক বাঁচাতে
    4. ইন্টারনেট না থাকলে

29. **MCQ 29.** সোর্স থেকে ইনস্টল বনাম প্যাকেজ ম্যানেজার — প্রোডাকশনে কোনটি সাধারণত ভালো?
    1. সোর্স সবসময়
    2. প্যাকেজ ম্যানেজার: আপডেট, নির্ভরতা ও অপসারণ সুশৃঙ্খল থাকে
    3. সোর্স দ্রুত
    4. কোনো পার্থক্য নেই

30. **MCQ 30.** systemd service-এ Restart=always দেওয়ার অর্থ কী?
    1. সার্ভিস কখনো থামে না
    2. ব্যর্থ হলে systemd আবার চালু করে
    3. boot-এ চালু হয়
    4. লগ মুছে যায়

31. **MCQ 31.** systemctl enable ও systemctl start-এর পার্থক্য কী?
    1. দুটোই এক
    2. enable boot-এ চালুর ব্যবস্থা করে, start এখনই চালায়
    3. start boot-এ চালায়
    4. enable এখনই চালায়

32. **MCQ 32.** journalctl দিয়ে লগ দেখার সুবিধা কী?
    1. লগ ছোট হয়
    2. সময়, ইউনিট ও অগ্রাধিকার ধরে ফিল্টার করা যায়
    3. লগ মুছে যায়
    4. ডিস্ক বাঁচে

33. **MCQ 33.** সার্ভিস চালু হচ্ছে না — প্রথমে কী দেখবেন?
    1. reboot
    2. systemctl status ও journalctl -u <service>
    3. ডিস্ক কেনা
    4. Nginx পুনরায় ইনস্টল

34. **MCQ 34.** Nginx-কে reverse proxy হিসেবে ব্যবহারের প্রধান কারণ কী?
    1. Node দ্রুত হয়
    2. static ফাইল, TLS ও লোড সামলানো Nginx-এর কাজ, অ্যাপ শুধু ব্যবসায়িক যুক্তিতে মনোযোগ দেয়
    3. ডেটাবেস দ্রুত হয়
    4. পোর্ট কম লাগে

35. **MCQ 35.** Nginx-এ proxy_pass-এ trailing slash-এর গুরুত্ব কী?
    1. কোনো গুরুত্ব নেই
    2. URI কীভাবে পেছনের অ্যাপে যাবে তা বদলে দিতে পারে
    3. গতি বাড়ায়
    4. TLS ঠিক করে

36. **MCQ 36.** location /api/ বনাম location = /api — পার্থক্য কী?
    1. দুটোই এক
    2. = শুধু ঠিক ওই পাথ মেলায়, prefix অনেক পাথ মেলায়
    3. prefix দ্রুত
    4. = ধীর

37. **MCQ 37.** Nginx কনফিগ বদলের পর নিরাপদ ক্রম কোনটি?
    1. সরাসরি restart
    2. nginx -t দিয়ে যাচাই, তারপর reload
    3. reboot
    4. ফাইল মুছে ফেলা

38. **MCQ 38.** 502 Bad Gateway-এর সবচেয়ে সম্ভাব্য অর্থ কী?
    1. Nginx বন্ধ
    2. Nginx চলছে কিন্তু পেছনের অ্যাপ সাড়া দিচ্ছে না
    3. DNS ভুল
    4. TLS মেয়াদ শেষ

39. **MCQ 39.** worker_processes auto দেওয়া সাধারণত কেন ভালো?
    1. RAM বাড়ে
    2. CPU কোর অনুযায়ী worker বসে

3. ডিস্ক বাড়ে
4. লগ কমে

40. **MCQ 40.** static ফাইল Nginx দিয়ে পরিবেশন করা Node দিয়ে করার চেয়ে ভালো কেন?
    1. Node ধীর
    2. Nginx static ফাইলে দক্ষ ও অ্যাপের CPU বাঁচায়
    3. Node static পারে না
    4. Nginx সস্তা

41. **MCQ 41.** দুটি ডোমেইন একই সার্ভারে চালাতে কী দরকার?
    1. দুটি সার্ভার
    2. প্রতিটি ডোমেইনের জন্য আলাদা server block / virtual host
    3. দুটি IP
    4. দুটি Nginx

42. **MCQ 42.** server_name ভুল থাকলে কী হয়?
    1. সার্ভার বন্ধ হয়
    2. অনুরোধ ভুল সাইটে (default) যেতে পারে
    3. TLS বন্ধ হয়
    4. ডেটাবেস বন্ধ হয়

43. **MCQ 43.** TLS সার্টিফিকেটের মেয়াদ শেষ হলে ব্যবহারকারী কী দেখবে?
    1. সাইট দ্রুত হবে
    2. ব্রাউজার নিরাপত্তা সতর্কতা দেখাবে
    3. সাইট মুছে যাবে
    4. 404 আসবে

44. **MCQ 44.** HTTP থেকে HTTPS-এ redirect কেন জরুরি?
    1. গতি বাড়াতে
    2. ব্যবহারকারী ভুল করে HTTP-তে এলে সুরক্ষিত সংযোগে পাঠাতে
    3. SEO কমাতে
    4. পোর্ট বাঁচাতে

45. **MCQ 45.** Let's Encrypt নবায়ন স্বয়ংক্রিয় রাখা কেন জরুরি?
    1. সার্টিফিকেট দামি
    2. ৯০ দিনের মেয়াদ ভুলে গেলে সাইট অবিশ্বস্ত হয়ে যায়
    3. নবায়ন কঠিন
    4. TLS ধীর হয়

46. **MCQ 46.** NODE_ENV=production না দিলে কী ঝুঁকি থাকে?
    1. অ্যাপ চলবে না
    2. ডেভেলপমেন্ট আচরণ, বিস্তারিত ত্রুটি ও ধীরগতি থাকতে পারে
    3. ডেটাবেস মুছে যায়
    4. পোর্ট বন্ধ হয়

47. **MCQ 47.** Node অ্যাপ সরাসরি পোর্ট 80-তে চালানো কেন সাধারণত এড়ানো হয়?
    1. Node 80 পারে না
    2. root অধিকার, TLS ও static সামলানো reverse proxy-র কাজ
    3. গতি কমে
    4. MongoDB বন্ধ হয়

48. **MCQ 48.** PM2-এর cluster mode কখন উপকারী?
    1. একটি CPU থাকলে
    2. বহু কোর ব্যবহার করে একই অ্যাপের একাধিক প্রসেস চালাতে
    3. ডেটাবেস দ্রুত করতে
    4. লগ কমাতে

49. **MCQ 49.** PM2 বনাম systemd — কখন systemd বেছে নেওয়া যুক্তিসঙ্গত?
    1. শুধু Node অ্যাপে
    2. সার্ভারের সব সার্ভিস একই ব্যবস্থায়, boot ও লগ একত্রে রাখতে
    3. শুধু cluster চাইলে
    4. PM2 না জানলে

50. **MCQ 50.** PM2-এ ecosystem ফাইল রাখার সুবিধা কী?
    1. অ্যাপ দ্রুত হয়
    2. ডেপ্লয় কনফিগ সংস্করণ-নিয়ন্ত্রণে ও পুনরুৎপাদনযোগ্য থাকে
    3. RAM বাড়ে
    4. TLS হয়

51. **MCQ 51.** PostgreSQL-এ connection pool বোঝা কেন জরুরি?
    1. ডেটাবেস দ্রুত হয়
    2. অনেক সংযোগ খুললে ডেটাবেস চাপে পড়ে; pool সীমা নিয়ন্ত্রণ করে
    3. ডিস্ক বাড়ে
    4. ব্যাকআপ হয়

52. **MCQ 52.** MongoDB-তে auth ছাড়া চালানো কেন বিপজ্জনক?
    1. ডেটাবেস ধীর হয়
    2. ইন্টারনেটে উন্মুক্ত থাকলে যে কেউ ডেটা পড়তে/মুছতে পারে

3. RAM বাড়ে
4. ব্যাকআপ বন্ধ হয়

53. **MCQ 53.** pg_dump নেওয়ার পর সবচেয়ে গুরুত্বপূর্ণ যাচাই কী?
    1. ফাইলের আকার দেখা
    2. restore করে দেখা যে ডেটা সত্যিই ফেরে
    3. ফাইল compress করা
    4. ফাইল মুছে ফেলা

54. **MCQ 54.** ডেটাবেস ব্যাকআপ অফ-সাইটে রাখা কেন জরুরি?
    1. ডিস্ক বাঁচাতে
    2. সার্ভার হারালে (আগুন/চুরি/ডিস্ক নষ্ট) ব্যাকআপও হারায়
    3. গতি বাড়াতে
    4. খরচ কমাতে

55. **MCQ 55.** MongoDB-র replica set কখন ভাববেন?
    1. একটি সার্ভার যথেষ্ট হলে
    2. উচ্চ প্রাপ্যতা ও স্বয়ংক্রিয় failover দরকার হলে
    3. ডিস্ক বাঁচাতে
    4. গতি কমাতে

56. **MCQ 56.** ডিস্ক 95% পূর্ণ — প্রথমে কী করবেন?
    1. বড় ফাইল মুছে ফেলা
    2. কী জায়গা নিচ্ছে তা শনাক্ত (du/df) করে নিরাপদে ব্যবস্থা নেওয়া
    3. reboot
    4. সার্ভার কেনা

57. **MCQ 57.** logrotate কেন জরুরি?
    1. লগ সুন্দর হয়
    2. লগ অসীম বেড়ে ডিস্ক ভরাট ঠেকায়
    3. লগ দ্রুত হয়
    4. লগ মুছে যায়

58. **MCQ 58.** du ও df-এর ফল না মিললে কী ভাববেন?
    1. একটি ভুল
    2. মুছে ফেলা কিন্তু খোলা ফাইল, বা মাউন্টের আড়ালে থাকা ফাইল
    3. ডিস্ক নষ্ট
    4. reboot দরকার

59. **MCQ 59.** free -h-তে available কম দেখালে কী বুঝবেন?
    1. RAM নষ্ট
    2. আসলেই ব্যবহারযোগ্য RAM কমছে — cache নয়, available-ই দেখতে হয়
    3. সার্ভার বন্ধ হবে
    4. swap শেষ

60. **MCQ 60.** swap বেশি ব্যবহার হওয়া কী নির্দেশ করে?
    1. সার্ভার ভালো চলছে
    2. RAM চাপে আছে — কারণ খোঁজা দরকার
    3. ডিস্ক দ্রুত
    4. CPU ভালো

61. **MCQ 61.** CPU iowait বেশি হলে কী বোঝা যায়?
    1. CPU নষ্ট
    2. CPU ডিস্কের জন্য অপেক্ষা করছে — ডিস্কই বাধা হতে পারে
    3. RAM শেষ
    4. নেটওয়ার্ক বন্ধ

62. **MCQ 62.** একটি অ্যাপ ধীর — CPU, RAM, ডিস্ক, নেটওয়ার্কের মধ্যে কীভাবে বাধা খুঁজবেন?
    1. অনুমান করে RAM বাড়ানো
    2. একেকটি মেট্রিক পরীক্ষা করে বাধাটি আলাদা করা
    3. reboot করা
    4. সব বাড়ানো

63. **MCQ 63.** ss -tulpn দিয়ে কী জানা যায়?
    1. CPU ব্যবহার
    2. কোন পোর্টে কোন প্রসেস শুনছে
    3. ডিস্ক ব্যবহার
    4. লগ

64. **MCQ 64.** DNS ঠিক না থাকলে প্রথম পরীক্ষা কোনটি?
    1. reboot
    2. nslookup/dig দিয়ে নাম মেলানো যায় কি না দেখা
    3. Nginx restart
    4. ফায়ারওয়াল বন্ধ

65. **MCQ 65.** ping কাজ করে কিন্তু curl ব্যর্থ — কী ভাববেন?
    1. সার্ভার বন্ধ
    2. নেটওয়ার্ক পথ আছে, কিন্তু অ্যাপ/পোর্ট/ফায়ারওয়াল স্তরে সমস্যা

3. DNS ভুল
4. ডিস্ক পূর্ণ

66. **MCQ 66.** UFW-তে default deny incoming কেন ভালো শুরু?
    1. সব বন্ধ থাকে
    2. শুধু প্রয়োজনীয় পোর্ট খোলা থাকে — আক্রমণের পথ কমে
    3. গতি বাড়ে
    4. SSH বন্ধ হয়

67. **MCQ 67.** ফায়ারওয়াল নিয়ম বদলের আগে কী সতর্কতা জরুরি?
    1. কিছুই না
    2. নিজেকে lockout করলে ফেরার পথ (console/বিকল্প) আছে কি না
    3. reboot
    4. ব্যাকআপ মুছে ফেলা

68. **MCQ 68.** nftables বনাম UFW — পার্থক্য কীভাবে দেখবেন?
    1. UFW দ্রুত
    2. UFW সহজ ইন্টারফেস, nftables নিচু স্তরের শক্তিশালী নিয়ন্ত্রণ
    3. nftables সহজ
    4. দুটোই এক

69. **MCQ 69.** একটি পোর্ট বাইরে থেকে খোলা দেখা যাচ্ছে কিন্তু ভেতর থেকে বন্ধ — কী হতে পারে?
    1. পোর্ট নষ্ট
    2. ফায়ারওয়াল/NAT স্তরে পার্থক্য, বা অ্যাপ শুধু নির্দিষ্ট ইন্টারফেসে শুনছে
    3. DNS ভুল
    4. TLS ভুল

70. **MCQ 70.** rate limiting Nginx-এ কেন বসাবেন?
    1. গতি বাড়াতে
    2. একটি IP-র অতিরিক্ত অনুরোধে অ্যাপ চাপে পড়া ঠেকাতে
    3. ডিস্ক বাঁচাতে
    4. TLS দ্রুত করতে

71. **MCQ 71.** fail2ban কী করে?
    1. ফায়ারওয়াল বদলায়
    2. বারবার ব্যর্থ লগইন দেখে IP সাময়িকভাবে বন্ধ করে
    3. পাসওয়ার্ড বদলায়
    4. SSH বন্ধ করে

72. **MCQ 72.** SSH-এ password authentication বন্ধ করার আগে কী নিশ্চিত করবেন?
    1. কিছুই না
    2. key দিয়ে লগইন সত্যিই কাজ করছে
    3. fail2ban আছে
    4. পোর্ট বদলানো

73. **MCQ 73.** অপ্রয়োজনীয় সার্ভিস বন্ধ করা কেন হার্ডেনিং?
    1. গতি বাড়ে
    2. প্রতিটি সার্ভিসই সম্ভাব্য আক্রমণের পথ; কম সার্ভিস মানে কম পথ
    3. ডিস্ক বাঁচে
    4. লগ কমে

74. **MCQ 74.** SUID ফাইল অডিট করা কেন জরুরি?
    1. গতি বাড়াতে
    2. SUID ভুল থাকলে সাধারণ ব্যবহারকারী root অধিকার পেতে পারে
    3. ডিস্ক বাঁচাতে
    4. ব্যাকআপ নিতে

75. **MCQ 75.** নিয়মিত নিরাপত্তা আপডেট না করলে কী হয়?
    1. সার্ভার ধীর হয়
    2. জানা দুর্বলতা খোলা থাকে — আক্রমণকারী সেটাই খোঁজে
    3. ডিস্ক ভরে
    4. SSH বন্ধ হয়

76. **MCQ 76.** একটি সার্ভিস শুধু localhost-এ শুনছে — এর সুবিধা কী?
    1. গতি বাড়ে
    2. বাইরে থেকে সরাসরি পৌঁছানো যায় না; reverse proxy হয়েই আসতে হয়
    3. ডিস্ক বাঁচে
    4. TLS লাগে না

77. **MCQ 77.** ব্যাকআপের 3-2-1 নীতির অর্থ কী?
    1. ৩টি সার্ভার
    2. ৩ কপি, ২ ভিন্ন মাধ্যম, ১ অফ-সাইট
    3. ৩ দিনের ব্যাকআপ
    4. ২টি ডিস্ক

78. **MCQ 78.** ব্যাকআপ নেওয়া আছে কিন্তু restore পরীক্ষা হয়নি — একে কী বলবেন?
    1. সম্পূর্ণ ব্যাকআপ
    2. অপরীক্ষিত ব্যাকআপ — প্রয়োজনে কাজ করবে তার প্রমাণ নেই

3. ব্যাকআপ নেই
4. নিরাপদ

79. **MCQ 79.** cron-এ একটি কাজ চলছে কি না কীভাবে নিশ্চিত হবেন?
    1. অনুমান করা
    2. লগ/আউটপুট ও সময়মতো ফল যাচাই করা
    3. reboot করা
    4. cron restart করা

80. **MCQ 80.** cron-এ PATH সমস্যা কেন হয়?
    1. cron ধীর
    2. cron-এর পরিবেশ ইন্টার‍্যাক্টিভ শেলের মতো নয়; পূর্ণ পাথ দরকার
    3. cron বন্ধ
    4. ডিস্ক পূর্ণ

81. **MCQ 81.** ব্যাকআপ স্ক্রিপ্টে set -e না থাকলে কী ঝুঁকি?
    1. স্ক্রিপ্ট দ্রুত হয়
    2. মাঝপথে ব্যর্থ হয়েও স্ক্রিপ্ট সফল দেখাতে পারে
    3. ডিস্ক ভরে
    4. লগ বাড়ে

82. **MCQ 82.** একটি cron কাজ প্রতিদিন ২টায় চলার কথা, কিন্তু চলছে না — প্রথমে কী দেখবেন?
    1. সার্ভার কেনা
    2. cron সার্ভিস চলছে কি না, সময়/টাইমজোন ও লগ
    3. Nginx restart
    4. ডিস্ক কেনা

83. **MCQ 83.** journald লগের retention সীমিত রাখা কেন জরুরি?
    1. লগ সুন্দর হয়
    2. লগ অসীম বেড়ে ডিস্ক ভরতে পারে
    3. লগ দ্রুত হয়
    4. SSH বন্ধ হয়

84. **MCQ 84.** error লগে 'connection refused' দেখলে কী বোঝা যায়?
    1. Nginx ভুল
    2. পেছনের সার্ভিস চলছে না বা ওই পোর্টে শুনছে না
    3. DNS ভুল
    4. TLS ভুল

85. **MCQ 85.** access লগে এক IP থেকে হাজার অনুরোধ — সঠিক প্রতিক্রিয়া কী?
    1. সঙ্গে সঙ্গে IP চিরতরে বন্ধ
    2. ধরন যাচাই করে rate limit / সাময়িক ব্লক ও মূল কারণ দেখা
    3. লগ মুছে ফেলা
    4. সার্ভার বন্ধ করা

86. **MCQ 86.** MERN ডেপ্লয়ে React build কোথায় পরিবেশন করা ভালো?
    1. Node-এর ভেতর দিয়ে সবসময়
    2. Nginx static হিসেবে, API Node-এ — দায়িত্ব ভাগ করে
    3. MongoDB দিয়ে
    4. সরাসরি পোর্ট 3000-এ

87. **MCQ 87.** API ও frontend একই ডোমেইনে /api পাথে রাখার সুবিধা কী?
    1. গতি বাড়ে
    2. CORS জটিলতা কমে ও এক TLS-এ সব চলে
    3. ডিস্ক বাঁচে
    4. ডেটাবেস দ্রুত হয়

88. **MCQ 88.** .env ফাইল git-এ চলে গেলে কী করবেন?
    1. ফাইল মুছে ফেলা
    2. secret ঘোরানো (rotate), git ইতিহাস থেকে সরানো, .gitignore ঠিক করা
    3. কিছুই না
    4. repo private করা

89. **MCQ 89.** প্রোডাকশনে secret পরিবেশ ভেরিয়েবলে রাখা ফাইলে hardcode করার চেয়ে ভালো কেন?
    1. দ্রুত হয়
    2. কোড ও secret আলাদা থাকে; repo-তে secret যায় না
    3. সস্তা
    4. সহজ

90. **MCQ 90.** ডেপ্লয়ের সময় ডেটাবেস মাইগ্রেশন আগে নাকি কোড আগে — কেন ভাবতে হয়?
    1. কোনো ব্যাপার নয়
    2. পুরোনো কোড নতুন স্কিমায় বা নতুন কোড পুরোনো স্কিমায় ভাঙতে পারে
    3. গতি বাড়ে
    4. ডিস্ক বাঁচে

91. **MCQ 91.** Blue-green ডেপ্লয়ের প্রধান সুবিধা কী?
    1. খরচ কমে
    2. নতুন সংস্করণ পরীক্ষা করে সুইচ করা যায়, ব্যর্থ হলে দ্রুত ফেরা যায়

3. গতি বাড়ে
4. ডিস্ক বাঁচে

92. **MCQ 92.** Rolling ডেপ্লয়ে কী সতর্কতা সবচেয়ে জরুরি?
    1. সব একসাথে বদলানো
    2. পুরোনো ও নতুন সংস্করণ একসঙ্গে চলাকালে সামঞ্জস্য রাখা
    3. reboot করা
    4. লগ মুছে ফেলা

93. **MCQ 93.** ডেপ্লয়ের পর smoke test কেন জরুরি?
    1. গতি বাড়াতে
    2. মূল পথগুলো (হোম, লগইন, API) সত্যিই কাজ করছে কি না দ্রুত যাচাই করতে
    3. লগ কমাতে
    4. ডিস্ক বাঁচাতে

94. **MCQ 94.** রোলব্যাক পরিকল্পনা না থাকলে ডেপ্লয় কেমন?
    1. সাহসী
    2. ঝুঁকিপূর্ণ — ব্যর্থ হলে ফেরার পথ নেই
    3. দ্রুত
    4. সস্তা

95. **MCQ 95.** ইনসিডেন্টে প্রথম কাজ কোনটি?
    1. দোষ খোঁজা
    2. সেবা ফেরানো (mitigate) ও প্রভাব মাপা
    3. পোস্টমর্টেম লেখা
    4. সবাইকে মেইল করা

96. **MCQ 96.** পোস্টমর্টেমে 'দোষারোপ নয়' নীতি কেন জরুরি?
    1. সবাই খুশি থাকে
    2. সত্য কারণ বের হয়; ভয়ে তথ্য লুকানো হয় না
    3. দ্রুত হয়
    4. সস্তা

97. **MCQ 97.** ইনসিডেন্টের সময়রেখা (timeline) কেন লেখা হয়?
    1. রিপোর্ট সুন্দর হয়
    2. কী কখন ঘটল তা প্রমাণসহ বোঝা যায় ও পুনরাবৃত্তি ঠেকানো যায়
    3. লগ কমে
    4. গতি বাড়ে

98. **MCQ 98.** সার্ভারে হঠাৎ সব সাইট ধীর — প্রথমে কোন মেট্রিক দেখবেন?
    1. ডোমেইনের দাম
    2. CPU, RAM, ডিস্ক I/O ও নেটওয়ার্ক — একসঙ্গে চিত্র নেওয়া
    3. TLS মেয়াদ
    4. ব্যাকআপ

99. **MCQ 99.** মনিটরিং alert-এ 'সবকিছুর alert' দেওয়া কেন খারাপ?
    1. alert বেশি ভালো
    2. সতর্কতা-ক্লান্তি (alert fatigue) হয় — আসল alert উপেক্ষিত হয়
    3. খরচ বাড়ে
    4. ডিস্ক ভরে

100. **MCQ 100.** সাপ্তাহিক রক্ষণাবেক্ষণে কী থাকা উচিত?
    1. শুধু reboot
    2. আপডেট যাচাই, ব্যাকআপ যাচাই, ডিস্ক/লগ পর্যালোচনা
    3. শুধু ব্যাকআপ
    4. শুধু লগ মুছে ফেলা

101. **MCQ 101.** একটি সার্ভিস বারবার restart হচ্ছে — সঠিক প্রতিক্রিয়া কী?
    1. Restart=always সরিয়ে দেওয়া
    2. লগে ব্যর্থতার কারণ খুঁজে ঠিক করা; restart শুধু লক্ষণ ঢাকে
    3. reboot করা
    4. সার্ভিস মুছে ফেলা

102. **MCQ 102.** প্রোডাকশন সার্ভারে পরীক্ষামূলক (experimental) প্যাকেজ ইনস্টল করা কেন ঝুঁকিপূর্ণ?
    1. গতি কমে
    2. অস্থিতিশীল নির্ভরতা সার্ভিস ভাঙতে পারে
    3. ডিস্ক ভরে
    4. SSH বন্ধ হয়

103. **MCQ 103.** ডেপ্লয় চেকলিস্ট থাকার মূল্য কী?
    1. কাগজপত্র বাড়ে
    2. চাপের মুহূর্তে ধাপ ভুলে যাওয়া ঠেকায় ও ডেপ্লয় পুনরুৎপাদনযোগ্য হয়
    3. গতি কমে
    4. খরচ বাড়ে

104. **MCQ 104.** সার্ভার unreachable — ping-ও ব্যর্থ — প্রথমে কী ভাববেন?
    1. অ্যাপ ভেঙেছে
    2. নেটওয়ার্ক/হোস্ট স্তরের সমস্যা — অ্যাপের আগে অবকাঠামো দেখা

3. Nginx ভুল
4. ডেটাবেস বন্ধ

105. **MCQ 105.** SSH লগইন ব্যর্থ কিন্তু সাইট চলছে — কী বোঝা যায়?
    1. সার্ভার বন্ধ
    2. SSH স্তরে সমস্যা (key, config, ফায়ারওয়াল); অ্যাপ আলাদা স্তর
    3. ডিস্ক পূর্ণ
    4. ডেটাবেস বন্ধ

106. **MCQ 106.** হঠাৎ ডিস্ক I/O বেড়ে গেলে কী করবেন?
    1. RAM বাড়ানো
    2. কোন প্রসেস I/O করছে তা শনাক্ত (iotop) করা
    3. CPU বাড়ানো
    4. reboot করা

107. **MCQ 107.** একটি API হঠাৎ 500 দিচ্ছে কিন্তু সাইটের static অংশ চলছে — কী ভাববেন?
    1. Nginx বন্ধ
    2. অ্যাপ/ডেটাবেস স্তরে সমস্যা; Nginx ও static ঠিক আছে
    3. DNS ভুল
    4. TLS ভুল

108. **MCQ 108.** ব্যাকআপ ফাইলের অখণ্ডতা (checksum) যাচাই কেন করবেন?
    1. ফাইল সুন্দর হয়
    2. নষ্ট ব্যাকআপ restore-এর সময় ধরা পড়ে, আগে নয় — আগে যাচাই দরকার
    3. গতি বাড়ে
    4. ডিস্ক বাঁচে

109. **MCQ 109.** সার্ভারে অজানা প্রসেস চলছে — প্রথমে কী করবেন?
    1. সঙ্গে সঙ্গে kill করা
    2. প্রসেসটি কী, কে চালাচ্ছে, কী করছে — শনাক্ত করা
    3. reboot করা
    4. সব মুছে ফেলা

110. **MCQ 110.** প্রোডাকশনে সবচেয়ে গুরুত্বপূর্ণ অভ্যাস কোনটি?
    1. দ্রুত কাজ করা
    2. পরিবর্তনের আগে ব্যাকআপ, পরে যাচাই — প্রতিটি পরিবর্তনে
    3. সব মনে রাখা
    4. reboot করা

111. **MCQ 111.** কোনো পরিবর্তনের পর সার্ভিস ঠিক আছে কি না কীভাবে নিশ্চিত হবেন?
    1. অনুমান করা
    2. status, লগ ও আসল অনুরোধ (curl/health check) দিয়ে যাচাই
    3. reboot করা
    4. কিছুক্ষণ অপেক্ষা করা

112. **MCQ 112.** সার্ভার ডকুমেন্টেশন (রানবুক) কেন জরুরি?
    1. কাগজপত্র বাড়ে
    2. আপনি না থাকলেও অন্য কেউ সার্ভার সামলাতে পারে; ইনসিডেন্টে সময় বাঁচে
    3. গতি বাড়ে
    4. খরচ কমে

113. **MCQ 113.** দীর্ঘমেয়াদে সার্ভার পরিচালনার সবচেয়ে বড় ঝুঁকি কোনটি?
    1. হার্ডওয়্যার দাম
    2. নথিভুক্ত না করা, যাচাই না করা ব্যাকআপ ও অনুমাননির্ভর পরিবর্তন
    3. ইন্টারনেট গতি
    4. ডোমেইন দাম

114. **MCQ 114.** একটি নতুন জুনিয়রকে সার্ভার অ্যাক্সেস দেওয়ার আগে কী করবেন?
    1. root দেওয়া
    2. ন্যূনতম অধিকার, key, ও কী করতে পারবে তার লিখিত সীমা ঠিক করা
    3. পাসওয়ার্ড শেয়ার করা
    4. কিছুই না

115. **MCQ 115.** ইনসিডেন্টের পর 'আবার যাতে না হয়' — সবচেয়ে কার্যকর পদক্ষেপ কী?
    1. সবাইকে সতর্ক করা
    2. মূল কারণের প্রতিরোধমূলক কাজ (action item) দায়িত্ব ও সময়সীমাসহ লেখা
    3. পোস্টমর্টেম মুছে ফেলা
    4. reboot করা

116. **MCQ 116.** প্রোডাকশন সার্ভার শেখার সবচেয়ে ভালো উপায় কী?
    1. শুধু পড়া
    2. ল্যাব VM-এ ভেঙে-গড়ে, লগ পড়ে ও ইনসিডেন্ট মহড়া দিয়ে শেখা
    3. ভিডিও দেখা
    4. নোট মুখস্থ করা

117. **MCQ 117.** সবশেষে — একটি ভালো প্রোডাকশন ইঞ্জিনিয়ারের পরিচয় কী?
    1. সব কমান্ড মুখস্থ
    2. পরিবর্তনের আগে ভাবে, যাচাই করে, নথিভুক্ত করে ও ব্যর্থতা থেকে শেখে

3. দ্রুত টাইপ করে
4. রাত জাগে

118. **MCQ 118.** সার্ভারে একটি নতুন প্যাকেজ ইনস্টলের আগে কী করা উচিত?
1. সরাসরি ইনস্টল করা
2. প্যাকেজটি কী করে, নির্ভরতা কী, প্রোডাকশনে প্রভাব কী — যাচাই করা
3. reboot করা
4. ব্যাকআপ মুছে ফেলা

119. **MCQ 119.** একটি সার্ভিসের লগে একই ত্রুটি বারবার আসছে — কী বোঝা যায়?
1. লগ ভুল
2. মূল কারণ ঠিক হয়নি; restart শুধু সাময়িক মুক্তি দিয়েছে
3. ডিস্ক ভুল
4. সার্ভিস ঠিক আছে

120. **MCQ 120.** প্রোডাকশনে পরিবর্তনের সবচেয়ে নিরাপদ ক্রম কোনটি?
1. পরিবর্তন → আশা করা
2. ব্যাকআপ → পরিবর্তন → যাচাই → নথিভুক্তকরণ
3. পরিবর্তন → reboot
4. পরিবর্তন → মুছে ফেলা

সংক্ষিপ্ত উত্তর (SA ১–৩০)

প্রতিটি উত্তর ২–৪ বাক্যে লিখুন।

1. **SA 1.** প্রোডাকশন সার্ভার বলতে কী বোঝায়? ডেভেলপমেন্ট পরিবেশ থেকে এর দুটি মূল পার্থক্য লিখুন।
2. **SA 2.** Ubuntu LTS বেছে নেওয়ার দুটি কারণ লিখুন।
3. **SA 3.** SSH key-based লগইন কীভাবে কাজ করে — দুই বাক্যে ব্যাখ্যা করুন।
4. **SA 4.** sudo-র ন্যূনতম অধিকার নীতি বলতে কী বোঝায়? একটি উদাহরণ দিন।
5. **SA 5.** LVM-এর দুটি সুবিধা লিখুন।
6. **SA 6.** inode শেষ হলে কী হয়? একটি বাস্তব উদাহরণ দিন।
7. **SA 7.** load average বলতে কী বোঝায়? 4 CPU-তে load 8 মানে কী?
8. **SA 8.** SIGTERM ও SIGKILL-এর পার্থক্য লিখুন।
9. **SA 9.** apt update ও apt upgrade-এর পার্থক্য লিখুন।
10. **SA 10.** systemctl enable ও systemctl start-এর পার্থক্য লিখুন।
11. **SA 11.** journalctl -u nginx -f কমান্ডটি কী করে? কখন ব্যবহার করবেন?
12. **SA 12.** Reverse proxy বলতে কী বোঝায়? একটি সুবিধা লিখুন।
13. **SA 13.** 502 Bad Gateway-এর দুটি সম্ভাব্য কারণ লিখুন।
14. **SA 14.** nginx -t ও nginx -s reload-এর ভূমিকা লিখুন।
15. **SA 15.** TLS সার্টিফিকেট নবায়ন স্বয়ংক্রিয় রাখা কেন জরুরি?
16. **SA 16.** NODE_ENV=production-এর দুটি প্রভাব লিখুন।
17. **SA 17.** PM2 ও systemd-এর একটি করে সুবিধা লিখুন।
18. **SA 18.** PostgreSQL-এ connection pool কেন দরকার?
19. **SA 19.** MongoDB-তে auth চালু না করলে কী ঝুঁকি?
20. **SA 20.** pg_dump-এর পর কী যাচাই করা উচিত এবং কেন?
21. **SA 21.** logrotate-এর কাজ কী? না থাকলে কী হয়?
22. **SA 22.** free -h-তে available বলতে কী বোঝায়?
23. **SA 23.** UFW-তে default deny নীতি কেন ভালো?
24. **SA 24.** fail2ban কীভাবে কাজ করে?
25. **SA 25.** 3-2-1 ব্যাকআপ নীতি ব্যাখ্যা করুন।
26. **SA 26.** cron-এ PATH সমস্যা কী? সমাধান কী?
27. **SA 27.** .env ফাইল git-এ গেলে কী করবেন? দুটি ধাপ লিখুন।
28. **SA 28.** Blue-green ডেপ্লয় কী? একটি সুবিধা লিখুন।
29. **SA 29.** Blameless পোস্টমর্টেম বলতে কী বোঝায়?
30. **SA 30.** একটি ভালো রানবুকে কী কী থাকা উচিত? তিনটি বিষয় লিখুন।

পরিস্থিতিভিত্তিক সমস্যা সমাধান (SC ১–২০)

প্রতিটি পরিস্থিতিতে আপনার করণীয় ধাপ, যুক্তি ও যাচাইয়ের উপায় লিখুন।

1. **SC 1.**

   সকাল ৯টায় technova.example খুলছে না, curl-এ 'connection refused' আসছে, কিন্তু সার্ভারে SSH হচ্ছে। প্রথম ৫ মিনিটে আপনি কী কী পরীক্ষা করবেন, কোন ক্রমে?

   প্রশ্ন: Nginx ও Node সার্ভিসের status, পোর্টে কে শুনছে (ss), লগের শেষ লাইন — এই ক্রমে।

2. **SC 2.**

   ডিস্ক 98% পূর্ণ, df-তে /var 40 GB দেখাচ্ছে। আপনি কীভাবে নিরাপদে জায়গা বের করবেন, কী মুছবেন না?

   প্রশ্ন: du দিয়ে বড় ডিরেক্টরি শনাক্ত, লগ rotate/compress, খোলা ফাইল যাচাই; ডেটাবেস/অ্যাপ ডেটা অনুমান করে মোছা নয়।

3. **SC 3.**

   একটি Node প্রসেস CPU 95% নিচ্ছে, সাইট ধীর। kill করার আগে কী করবেন?

   প্রশ্ন: প্রসেস শনাক্ত, লগ/মেট্রিক দেখা, infinite loop না ট্রাফিক-চাপ বোঝা, প্রয়োজনে graceful restart ও কারণ নথিভুক্ত করা।

4. **SC 4.**

   ডেপ্লয়ের পর সব API 502 দিচ্ছে, static পাতা চলছে। কারণ খোঁজার ধাপগুলো লিখুন।

   প্রশ্ন: Nginx error লগ, upstream পোর্টে অ্যাপ চলছে কি না, env/পোর্ট বদলেছে কি না, শেষ ডেপ্লয়ের পরিবর্তন।

5. **SC 5.**

   SSH-এ 'Permission denied (publickey)' আসছে। সাইট চলছে। ধাপে ধাপে কী পরীক্ষা করবেন?

   প্রশ্ন: key ফাইল, অনুমতি (700/600), authorized_keys, sshd_config, ফায়ারওয়াল — সার্ভারে console/alternate পথে।

6. **SC 6.**

   PostgreSQL-এ 'too many connections' ত্রুটি আসছে। তাৎক্ষণিক ও দীর্ঘমেয়াদি সমাধান কী?

   প্রশ্ন: তাৎক্ষণিক: অপ্রয়োজনীয় সংযোগ বন্ধ/অ্যাপ restart; দীর্ঘমেয়াদি: pool সীমা, pooler, সংযোগ লিক ঠিক করা।

7. **SC 7.**

   সার্টিফিকেটের মেয়াদ কাল শেষ হবে, নবায়ন ব্যর্থ হচ্ছে। আপনি কী করবেন?

   প্রশ্ন: certbot লগ দেখা, পোর্ট 80/চ্যালেঞ্জ পথ যাচাই, dry-run, প্রয়োজনে হাতে নবায়ন ও মনিটরিং alert বসানো।

8. **SC 8.**

   cron-এ রাত ২টার ব্যাকআপ চলছে না, লগে কিছু নেই। কী পরীক্ষা করবেন?

প্রশ্ন: cron সার্ভিস, crontab সিনট্যাক্স, টাইমজোন, PATH/পূর্ণ পাথ, স্ক্রিপ্টের অনুমতি ও আউটপুট রিডাইরেক্ট।

9. **SC 9.**

   সার্ভার reboot-এর পর Node অ্যাপ চালু হয়নি, Nginx চলছে। কারণ কী হতে পারে?

   প্রশ্ন: সার্ভিস enable করা হয়নি, EnvironmentFile পাথ ভুল, ডেটাবেস তখনো ওঠেনি (After=/Requires)।

10. **SC 10.**

    MongoDB-তে হঠাৎ ডিস্ক দ্রুত ভরছে। কী খুঁজবেন?

    প্রশ্ন: oplog/লগ আকার, অপ্রয়োজনীয় সংগ্রহ, বড় ডকুমেন্ট, index আকার — du ও db.stats দিয়ে।

11. **SC 11.**

    এক IP থেকে প্রতি সেকেন্ডে শত শত অনুরোধ আসছে, সাইট ধীর। আপনার প্রতিক্রিয়া কী?

    প্রশ্ন: লগে ধরন যাচাই (বৈধ বট না আক্রমণ), rate limit, সাময়িক ব্লক, upstream চাপ পরীক্ষা।

12. **SC 12.**

    apt upgrade-এর পর Nginx চালু হচ্ছে না। কী করবেন?

    প্রশ্ন: journalctl/status, nginx -t, কনফিগ পরিবর্তন/ব্যাকআপ থেকে তুলনা, প্রয়োজনে আগের সংস্করণে ফেরা।

13. **SC 13.**

    সাইট HTTPS-এ 'Not Secure' দেখাচ্ছে। সম্ভাব্য কারণগুলো কী?

    প্রশ্ন: মেয়াদোত্তীর্ণ/ভুল ডোমেইনের সার্টিফিকেট, mixed content, redirect লুপ।

14. **SC 14.**

    ফায়ারওয়াল নিয়ম বদলের পর SSH বন্ধ হয়ে গেছে। ফেরার পথ কী?

    প্রশ্ন: হোস্টিং console/VNC, বিকল্প পোর্ট/ব্যবহারকারী, নিয়ম ফেরানো — আগে থেকে ফেরার পথ রাখা উচিত ছিল।

15. **SC 15.**

    RAM 90%+ ব্যবহার হচ্ছে, swap-ও বাড়ছে। কী করবেন?

    প্রশ্ন: কোন প্রসেস RAM নিচ্ছে তা শনাক্ত, memory leak না ট্রাফিক-চাপ বোঝা, restart নয় — কারণ ঠিক করা।

16. **SC 16.**

    ডেপ্লয়ের পর কিছু ব্যবহারকারী পুরোনো সংস্করণ দেখছে। কেন?

    প্রশ্ন: ক্যাশ (browser/CDN/Nginx), rolling-এ দুই সংস্করণ, load balancer-এ পুরোনো নোড।

17. **SC 17.**

    লগে 'No space left on device' কিন্তু df-তে জায়গা আছে। কী হতে পারে?

    প্রশ্ন: inode শেষ, বা মুছে ফেলা খোলা ফাইল, বা /tmp-এর আলাদা সীমা।

18. **SC 18.**

    ডেটাবেস restore করতে গিয়ে দেখলেন ব্যাকআপ ফাইল নষ্ট। এখন কী?

    প্রশ্ন: অন্য কপি (অফ-সাইট/পুরোনো) খোঁজা, ঘটনা নথিভুক্ত করা, ভবিষ্যতে checksum ও restore পরীক্ষা বাধ্যতামূলক করা।

19. **SC 19.**

    ইনসিডেন্ট চলছে, ম্যানেজার প্রতি ৫ মিনিটে আপডেট চাইছেন। আপনি কীভাবে যোগাযোগ সামলাবেন?

    প্রশ্ন: একজন যোগাযোগকারী ঠিক করা, নির্দিষ্ট সময়ে সংক্ষিপ্ত আপডেট, কাজের ধারা না ভাঙা।

20. **SC 20.**

    সার্ভারে অজানা প্রসেস ও অদ্ভুত নেটওয়ার্ক সংযোগ দেখা যাচ্ছে। প্রথম পদক্ষেপ কী?

    প্রশ্ন: প্রসেস/সংযোগ শনাক্ত, নেটওয়ার্ক আলাদা করা, প্রমাণ সংরক্ষণ, অনুমান করে মুছে ফেলা নয়।

প্রোডাকশন ইনসিডেন্ট কেস স্টাডি (CS ১–১০)

প্রতিটি ঘটনা পড়ে মূল কারণ, তাৎক্ষণিক করণীয় ও দীর্ঘমেয়াদি প্রতিরোধ — তিনটি অংশে বিশ্লেষণ লিখুন।

1. **CS 1: ডিস্ক পূর্ণে সাইট বন্ধ**

   শুক্রবার রাত ১১টায় technova.example বন্ধ। ডিস্ক 100% — /var/log/nginx-এ 30 GB লগ, logrotate ভুল কনফিগে চলেনি। টিম লগ মুছে সাইট চালু করে, কিন্তু দুই সপ্তাহ পর আবার একই ঘটনা।

   বিশ্লেষণ প্রশ্ন: মূল কারণ কী? মুছে ফেলা কেন স্থায়ী সমাধান নয়? logrotate ঠিক করা, মনিটরিং alert ও retention নীতি — কী কী করবেন?

2. **CS 2: ভুল ডেপ্লয়ে 502**

   সোমবার সকালে ডেপ্লয়ের পর সব API 502। দেখা গেল নতুন কোডে PORT env বদলে গেছে, কিন্তু Nginx-এ পুরোনো পোর্ট। রোলব্যাকের কোনো লিখিত ধাপ ছিল না, ৪০ মিনিট সাইট বন্ধ।

   বিশ্লেষণ প্রশ্ন: কোথায় ভুল হয়েছিল? ডেপ্লয় চেকলিস্টে কী যোগ করবেন? রোলব্যাক কীভাবে ৫ মিনিটে নামাবেন?

3. **CS 3: ফাঁস হওয়া secret**

   একজন ডেভেলপার .env ফাইল ভুলে git-এ পুশ করেন। দুই দিন পর কেউ ডেটাবেসে অচেনা লগইন দেখে।

   বিশ্লেষণ প্রশ্ন: তাৎক্ষণিক কী করবেন? secret rotate, ইতিহাস পরিষ্কার, .gitignore ও pre-commit যাচাই — ধাপগুলো লিখুন।

4. **CS 4: SSH lockout**

   একজন অ্যাডমিন পাসওয়ার্ড লগইন বন্ধ করেন key পরীক্ষা না করে। পরের দিন কেউ সার্ভারে ঢুকতে পারছে না, সাইট চলছে।

   বিশ্লেষণ প্রশ্ন: ভুলটি কী? ফেরার পথ কী? ভবিষ্যতে এই ভুল ঠেকাতে কী নীতি নেবেন?

5. **CS 5: ব্যাকআপ যা ফেরে না**

   ডিস্ক নষ্টের পর টিম ব্যাকআপ restore করতে গিয়ে দেখে গত ৩ মাসের ব্যাকআপ ফাইল নষ্ট — কেউ restore পরীক্ষা করেনি।

   বিশ্লেষণ প্রশ্ন: এই ঘটনা থেকে কী শিখবেন? ব্যাকআপ নীতিতে কী বদলাবেন (পরীক্ষা, checksum, অফ-সাইট)?

6. **CS 6: CPU-ক্ষুধার্ত প্রসেস**

   একটি ব্যাচ জব প্রতিদিন দুপুরে CPU 100% নিয়ে সাইট ধীর করে। টিম প্রতিদিন সার্ভার reboot করে।

   বিশ্লেষণ প্রশ্ন: reboot কেন সমাধান নয়? আসল কারণ খুঁজতে কী করবেন? সময়সূচি/সীমা (nice, সময় বদল) কীভাবে ঠিক করবেন?

7. **CS 7: TLS মেয়াদ শেষ**

   ছুটির দিনে সার্টিফিকেটের মেয়াদ শেষ, সাইটে ব্রাউজার সতর্কতা। নবায়ন স্বয়ংক্রিয় ছিল, কিন্তু কেউ alert দেখেনি।

   বিশ্লেষণ প্রশ্ন: কেন স্বয়ংক্রিয় নবায়নও ব্যর্থ হতে পারে? মনিটরিং ও দায়িত্বে কী যোগ করবেন?

8. **CS 8: ডেটাবেস সংযোগ শেষ**

   প্রচারের দিন ট্রাফিক বেড়ে PostgreSQL-এ 'too many connections'। অ্যাপ প্রতিটি অনুরোধে নতুন সংযোগ খুলছিল।

   বিশ্লেষণ প্রশ্ন: মূল কারণ কী? pool, pooler ও কোডের সংযোগ ব্যবস্থাপনায় কী বদলাবেন?

9. **CS 9: ফায়ারওয়াল ভুল**

   UFW-তে ভুল নিয়মে পোর্ট 443 বন্ধ হয়ে যায়, সাইট HTTPS-এ বন্ধ। টিম ভাবে অ্যাপ ভেঙেছে, ১ ঘণ্টা অ্যাপ খোঁজে।

   বিশ্লেষণ প্রশ্ন: তদন্তের ক্রমে কী ভুল ছিল? স্তর ধরে (নেটওয়ার্ক→ফায়ারওয়াল→Nginx→অ্যাপ) পরীক্ষার গুরুত্ব ব্যাখ্যা করুন।

10. **CS 10: ইনসিডেন্টে যোগাযোগ ভাঙন**

    ইনসিডেন্টে পাঁচজন আলাদা আলাদা কাজ করে, কেউ জানে না কে কী করছে; ব্যবহারকারীদের কোনো আপডেট নেই।

    বিশ্লেষণ প্রশ্ন: যোগাযোগের কাঠামো (একজন সমন্বয়কারী, সময়রেখা, আপডেট ছক) কেমন হওয়া উচিত? পোস্টমর্টেমে কী লিখবেন?

চূড়ান্ত ব্যবহারিক পরীক্ষা (Final Examination)

মোট নম্বর: ১০০ | সময়: ৪ ঘণ্টা | পাস নম্বর: ৬০ — তত্ত্বে অন্তত ২০/৪০ ও ব্যবহারিকে অন্তত ৩০/৬০ পেতে হবে; এক অংশে পাস করলেই মোট পাস হয় না। পরীক্ষা ল্যাব VM-এ হবে।

ক — তত্ত্ব (৪০ নম্বর, ৯০ মিনিট)

অংশ	নম্বর	সময়
MCQ (২০টি, বইয়ের ২৪ অধ্যায় থেকে)	২০	৩০ মিনিট
সংক্ষিপ্ত উত্তর (৫টি × ৪)	২০	৬০ মিনিট

খ — হাতে-কলমে (৬০ নম্বর, ২ ঘণ্টা ৩০ মিনিট)

কাজ	নম্বর	যা যাচাই হবে
১. Ubuntu সার্ভারে SSH হার্ডেনিং (key login, root বন্ধ, UFW)	১২	key ছাড়া লগইন ব্যর্থ, root বন্ধ, শুধু প্রয়োজনীয় পোর্ট খোলা
২. Nginx reverse proxy + TLS (app.technova.example)	১২	nginx -t সফল, HTTPS redirect, proxy header সঠিক
৩. Node অ্যাপ systemd সার্ভিস হিসেবে চালু + লগ যাচাই	১২	reboot-পরবর্তী চালু, restart নীতি, journald লগ
৪. PostgreSQL বা MongoDB ব্যাকআপ নিয়ে restore পরীক্ষা	১২	restore-এর প্রমাণ, auth যাচাই
৫. 'ডিস্ক পূর্ণ' ও '৫০২' — দুটি ইনসিডেন্ট নির্ণয় ও সমাধান	১২	মূল কারণ শনাক্ত, নিরাপদ সমাধান, নথিভুক্তকরণ

> **সতর্কতা:** পরীক্ষার ধ্বংসাত্মক ধাপ শুধু নির্ধারিত ল্যাব VM-এ করবেন; পরীক্ষকের নির্দেশ ছাড়া প্রোডাকশন-সদৃশ ডেটা মুছবেন না।

পাসের শর্ত: মোট ৬০+ এবং তত্ত্বে ২০+ ও ব্যবহারিকে ৩০+। কোনো অংশে শর্ত না মিললে অকৃতকার্য ধরা হবে, মোট বেশি হলেও।

ক্যাপস্টোন মূল্যায়ন রুব্রিক

ক্যাপস্টোন (অ্যাসাইনমেন্ট ২০) এই মানদণ্ডে মূল্যায়ন করা হবে। প্রতিটি মানদণ্ডে প্রমাণ (লগ, আউটপুট, স্ক্রিনশট) ছাড়া নম্বর দেওয়া হবে না।

মানদণ্ড	উত্তীর্ণ (পাস)	ভালো	চমৎকার	নম্বর
SSH ও ব্যবহারকারী সুরক্ষা	key login চালু, root বন্ধ	+ sudo সীমিত, fail2ban-সদৃশ সুরক্ষা	+ অডিট লগ ও নিয়মিত পর্যালোচনা	১০
Nginx + TLS	reverse proxy ও HTTPS চালু	+ redirect, HSTS, স্বয়ংক্রিয় নবায়ন	+ rate limit ও নিরাপত্তা হেডার	
অ্যাপ ডেপ্লয় (systemd/PM2)	অ্যাপ চলে, reboot-এ ফেরে	+ health check, সঠিক restart নীতি	+ blue-green/rolling পরিকল্পনা ও রোলব্যাক মহড়া	১৫
ডেটাবেস ও ব্যাকআপ	ব্যাকআপ ফাইল আছে	+ restore পরীক্ষার প্রমাণ	+ অফ-সাইট কপি, checksum, সময়সূচি	১৫
হার্ডেনিং ও ফায়ারওয়াল	UFW-তে শুধু প্রয়োজনীয় পোর্ট	+ অপ্রয়োজনীয় সার্ভিস বন্ধ, অডিট	+ নিয়মিত অডিট প্রতিবেদন	১০
মনিটরিং, লগ ও রক্ষণাবেক্ষণ	লগ দেখা যায়	+ alert, logrotate, ক্যালেন্ডার	+ ড্যাশবোর্ড ও ইনসিডেন্ট মহড়া	১০
রানবুক ও নথিভুক্তকরণ	মূল ধাপ লেখা	+ ইনসিডেন্ট পদ্ধতি ও যোগাযোগ	+ পোস্টমর্টেম টেমপ্লেট ও পরীক্ষিত রোলব্যাক	১০
ইনসিডেন্ট মহড়া	একটি মহড়া সম্পন্ন	+ সময়রেখা ও মূল কারণ	+ প্রতিরোধমূলক কাজ দায়িত্বসহ	১০

মোট: ১০০ নম্বর; পাস: ৬০। প্রতিটি মানদণ্ডে 'উত্তীর্ণ' স্তর না পেলে সেই মানদণ্ডে শূন্য ধরা হবে।

উত্তর নির্দেশিকা (Answer Key) — সব প্রশ্নের পরে, আলাদা অংশ

এই অংশটি প্রশ্নের পরে ইচ্ছাকৃতভাবে আলাদা রাখা হয়েছে। আগে নিজে চেষ্টা করুন, তারপর মিলিয়ে নিন।

MCQ উত্তর ও ব্যাখ্যা (MCQ ১–১২০)

1. **MCQ 1 — উত্তর: B.** প্রোডাকশনে ধারাবাহিকতা ও ব্যর্থতার প্রভাবই মূল পার্থক্য, হার্ডওয়্যার নয়।
2. **MCQ 2 — উত্তর: B.** LTS-এর মূল্য দীর্ঘ সমর্থন ও স্থিতিশীলতায়, নতুনত্বে নয়।
3. **MCQ 3 — উত্তর: B.** /var ভরে গেলে root ভরে যাওয়া ঠেকানোই আলাদা পার্টিশনের মূল উদ্দেশ্য।
4. **MCQ 4 — উত্তর: C.** swap একটি নকশা-সিদ্ধান্ত, কাজের ধরন ও RAM অনুযায়ী নিতে হয়।
5. **MCQ 5 — উত্তর: B.** আপডেট ছাড়া ডেপ্লয় করলে জানা দুর্বলতা নিয়ে সার্ভার চালু হয়।
6. **MCQ 6 — উত্তর: B.** দীর্ঘ ক্রিপ্টোগ্রাফিক key অনুমান-আক্রমণের বিরুদ্ধে অনেক শক্তিশালী।
7. **MCQ 7 — উত্তর: B.** সরাসরি root লগইন জবাবদিহি নষ্ট করে ও আক্রমণকারীর লক্ষ্য সহজ করে।
8. **MCQ 8 — উত্তর: B.** ন্যূনতম অধিকারের নীতি অনুযায়ী শুধু প্রয়োজনীয় কমান্ডের sudo-ই সঠিক।
9. **MCQ 9 — উত্তর: B.** sudo লগ জবাবদিহি ও ইনসিডেন্ট তদন্তের ভিত্তি।
10. **MCQ 10 — উত্তর: B.** পোর্ট বদলানো obscurity মাত্র, আসল সুরক্ষা প্রমাণীকরণ ও নিয়ন্ত্রণে।
11. **MCQ 11 — উত্তর: B.** LVM-এর মূল্য নমনীয় ভলিউম ব্যবস্থাপনায়।
12. **MCQ 12 — উত্তর: B.** মনিটরিং ছাড়া পূর্ণ হওয়া টের পাওয়া যায় ঘটনার পর, আগে নয়।
13. **MCQ 13 — উত্তর: B.** খোলা ফাইল মুছলে inode মুক্ত হয় না যতক্ষণ প্রসেসটি ধরে রাখে।
14. **MCQ 14 — উত্তর: B.** inode শেষ হলে ফাইল তৈরির স্লট থাকে না, জায়গা থাকলেও।
15. **MCQ 15 — উত্তর: B.** ছোট ফাইল বেশি হলে inode-ই প্রথম শেষ হয়, বাইট নয়।
16. **MCQ 16 — উত্তর: B.** 777 মানে সবার পড়া-লেখা-চালানো — ন্যূনতম অধিকারের সম্পূর্ণ উল্টো।
17. **MCQ 17 — উত্তর: B.** গোপন তথ্যের ফাইল শুধু মালিকের পড়ার মতো সীমিত রাখাই সঠিক।
18. **MCQ 18 — উত্তর: C.** দুই লিংকের মুছে ফেলা ও ব্যাকআপ আচরণ আলাদা, ভুল করলে ডেটা হারাতে পারে।
19. **MCQ 19 — উত্তর: B.** লক্ষণ মুছলে সাময়িক মুক্তি মেলে, মূল কারণ ঠিক না করলে আবার জমবে।
20. **MCQ 20 — উত্তর: B.** fsck একটি মেরামতের হাতিয়ার, নিয়মিত রুটিন নয়; ভুল ব্যবহারে ক্ষতি হতে পারে।
21. **MCQ 21 — উত্তর: B.** CPU সংখ্যার চেয়ে বেশি load মানে কাজ জমে আছে।
22. **MCQ 22 — উত্তর: B.** kill করার আগে শনাক্তকরণ জরুরি, নইলে গুরুত্বপূর্ণ প্রসেস মরতে পারে।
23. **MCQ 23 — উত্তর: B.** zombie হলো সমাপ্ত সন্তান যার status প্যারেন্ট এখনো নেয়নি।
24. **MCQ 24 — উত্তর: B.** প্রথমে SIGTERM, কাজ না হলে SIGKILL — এটাই সভ্য ক্রম।
25. **MCQ 25 — উত্তর: B.** nice বেশি মানে CPU পাওয়ার অগ্রাধিকার কম।
26. **MCQ 26 — উত্তর: B.** update শুধু সূচি আনে, upgrade আসল ইনস্টল করে।
27. **MCQ 27 — উত্তর: B.** স্বয়ংক্রিয় আপডেট সুবিধাজনক হলেও restart/পরিবর্তনের ঝুঁকি থাকে, তাই নীতি দরকার।
28. **MCQ 28 — উত্তর: B.** সামঞ্জস্য পরীক্ষা ছাড়া আপগ্রেড ঝুঁকিপূর্ণ হলে hold যুক্তিসঙ্গত, তবে নিরাপত্তা প্যাচের ঝুঁকিও দেখতে হয়।

29. **MCQ 29 — উত্তর: B.** প্যাকেজ ম্যানেজারে রক্ষণাবেক্ষণ ও আপডেটের শৃঙ্খলা থাকে।
30. **MCQ 30 — উত্তর: B.** Restart=always ব্যর্থতায় পুনরায় চালুর নীতি, অমরত্বের গ্যারান্টি নয়।
31. **MCQ 31 — উত্তর: B.** enable স্থায়ী (boot) ব্যবস্থা, start তাৎক্ষণিক চালু।
32. **MCQ 32 — উত্তর: B.** journald-এর কাঠামোবদ্ধ লগে সময় ও ইউনিট ধরে খোঁজা যায়।
33. **MCQ 33 — উত্তর: B.** status ও লগেই ব্যর্থতার কারণ লেখা থাকে, অনুমান নয়।
34. **MCQ 34 — উত্তর: B.** দায়িত্ব ভাগ করাই reverse proxy-র মূল্য।
35. **MCQ 35 — উত্তর: B.** trailing slash URI ম্যাপিং বদলায়, তাই ভুল হলে 404 আসে।
36. **MCQ 36 — উত্তর: B.** exact match ও prefix match-এর আচরণ আলাদা।
37. **MCQ 37 — উত্তর: B.** পরীক্ষা ছাড়া restart করলে ভুল কনফিগে সার্ভিস থেমে যেতে পারে; reload সংযোগ ভাঙে না।
38. **MCQ 38 — উত্তর: B.** 502 মানে proxy পেছনের সার্ভার থেকে বৈধ সাড়া পায়নি।
39. **MCQ 39 — উত্তর: B.** auto কোর সংখ্যার সঙ্গে worker মেলায়।
40. **MCQ 40 — উত্তর: B.** static কাজ proxy-তে রেখে অ্যাপের CPU ব্যবসায়িক কাজে ব্যবহার করা যায়।
41. **MCQ 41 — উত্তর: B.** virtual host-ই এক সার্ভারে বহু সাইটের উপায়।
42. **MCQ 42 — উত্তর: B.** Host না মিললে Nginx default server-এ পাঠায়।
43. **MCQ 43 — উত্তর: B.** মেয়াদোত্তীর্ণ সার্টিফিকেটে ব্রাউজার বিশ্বাস করে না।
44. **MCQ 44 — উত্তর: B.** redirect ছাড়া HTTP পথ খোলা থাকে।
45. **MCQ 45 — উত্তর: B.** স্বল্প মেয়াদের সার্টিফিকেটে স্বয়ংক্রিয় নবায়নই ভরসা।
46. **MCQ 46 — উত্তর: B.** production মোড ছাড়া অ্যাপ ডেভ আচরণে চলতে পারে।
47. **MCQ 47 — উত্তর: B.** privileged পোর্ট ও প্রান্তিক দায়িত্ব proxy-তে রাখাই নিরাপদ।
48. **MCQ 48 — উত্তর: B.** cluster একাধিক কোরে অ্যাপ ছড়িয়ে দেয়।
49. **MCQ 49 — উত্তর: B.** systemd সার্ভারব্যাপী একক সার্ভিস ব্যবস্থা দেয়।
50. **MCQ 50 — উত্তর: B.** কনফিগ ফাইলে থাকলে ডেপ্লয় পুনরুৎপাদনযোগ্য হয়।
51. **MCQ 51 — উত্তর: B.** অনিয়ন্ত্রিত সংযোগ ডেটাবেসকে ক্লান্ত করে।
52. **MCQ 52 — উত্তর: B.** auth ছাড়া উন্মুক্ত MongoDB সরাসরি ডেটা-চুরির আমন্ত্রণ।
53. **MCQ 53 — উত্তর: B.** restore পরীক্ষা ছাড়া ব্যাকআপ কেবল আশা, প্রমাণ নয়।
54. **MCQ 54 — উত্তর: B.** একই সার্ভারের ব্যাকআপ সার্ভারের সঙ্গেই হারায়।
55. **MCQ 55 — উত্তর: B.** replica set-এর মূল্য প্রাপ্যতা ও failover-এ।
56. **MCQ 56 — উত্তর: B.** শনাক্ত না করে মুছলে গুরুত্বপূর্ণ ডেটা যেতে পারে।
57. **MCQ 57 — উত্তর: B.** logrotate ছাড়া লগই একদিন ডিস্ক ভরিয়ে ফেলে।
58. **MCQ 58 — উত্তর: B.** খোলা-মুছে-ফেলা ফাইল ও মাউন্ট আড়ালই সাধারণ কারণ।
59. **MCQ 59 — উত্তর: B.** available-ই আসল ব্যবহারযোগ্য RAM, total used-এ cache ধরা থাকে।
60. **MCQ 60 — উত্তর: B.** ঘন ঘন swap মানে RAM চাপ, স্বাভাবিক অবস্থা নয়।
61. **MCQ 61 — উত্তর: B.** iowait বেশি মানে CPU ডিস্ক I/O-র অপেক্ষায়।
62. **MCQ 62 — উত্তর: B.** পদ্ধতিগত পরীক্ষা ছাড়া বাধা শনাক্ত হয় না।
63. **MCQ 63 — উত্তর: B.** ss listening পোর্ট ও প্রসেস দেখায়।
64. **MCQ 64 — উত্তর: B.** DNS সমস্যা DNS পরীক্ষাতেই ধরা পড়ে।

65. **MCQ 65 — উত্তর: B.** ping ICMP, curl TCP/HTTP — স্তর আলাদা, তাই কারণও আলাদা।
66. **MCQ 66 — উত্তর: B.** deny-by-default ন্যূনতম উন্মুক্তির নীতি।
67. **MCQ 67 — উত্তর: B.** SSH বন্ধ হয়ে গেলে ফেরার পথ না থাকলে সার্ভার অচল।
68. **MCQ 68 — উত্তর: B.** UFW সরলতা দেয়, nftables সূক্ষ্ম নিয়ন্ত্রণ দেয়।
69. **MCQ 69 — উত্তর: B.** listen ঠিকানা ও ফায়ারওয়াল — দুটোই পোর্টের দৃশ্যমানতা বদলায়।
70. **MCQ 70 — উত্তর: B.** rate limit অপব্যবহার ও আকস্মিক চাপ ঠেকায়।
71. **MCQ 71 — উত্তর: B.** fail2ban লগ দেখে পুনরাবৃত্ত আক্রমণকারীকে থামায়।
72. **MCQ 72 — উত্তর: B.** key পরীক্ষা ছাড়া পাসওয়ার্ড বন্ধ করলে নিজেই আটকে যাবেন।
73. **MCQ 73 — উত্তর: B.** আক্রমণের পৃষ্ঠতল কমানোই হার্ডেনিংয়ের মূল।
74. **MCQ 74 — উত্তর: B.** SUID অধিকার-বৃদ্ধির পথ, তাই নিয়মিত যাচাই দরকার।
75. **MCQ 75 — উত্তর: B.** জানা দুর্বলতাই সবচেয়ে সহজ আক্রমণপথ।
76. **MCQ 76 — উত্তর: B.** localhost bind অভ্যন্তরীণ সার্ভিসকে বাইরের থেকে আড়াল করে।
77. **MCQ 77 — উত্তর: B.** 3-2-1 কপি, মাধ্যম ও অবস্থানের বৈচিত্র্য চায়।
78. **MCQ 78 — উত্তর: B.** পরীক্ষা ছাড়া ব্যাকআপের ভরসা নেই।
79. **MCQ 79 — উত্তর: B.** cron নীরবে ব্যর্থ হতে পারে; লগ ও ফলই প্রমাণ।
80. **MCQ 80 — উত্তর: B.** cron-এ সংক্ষিপ্ত কমান্ড PATH-এর কারণে ব্যর্থ হতে পারে।
81. **MCQ 81 — উত্তর: B.** set -e ছাড়া ব্যর্থ ধাপ চুপচাপ এগিয়ে যায়।
82. **MCQ 82 — উত্তর: B.** cron সার্ভিস, সময় ও লগ — এই তিনটিই প্রথম পরীক্ষা।
83. **MCQ 83 — উত্তর: B.** journald-এর আকার সীমা না থাকলে ডিস্ক ভরে।
84. **MCQ 84 — উত্তর: B.** connection refused মানে লক্ষ্য পোর্টে কেউ শুনছে না।
85. **MCQ 85 — উত্তর: B.** আগে যাচাই — বৈধ ট্রাফিকও হতে পারে, আক্রমণও হতে পারে।
86. **MCQ 86 — উত্তর: B.** static Nginx-এ, API Node-এ — এটাই দক্ষ ভাগ।
87. **MCQ 87 — উত্তর: B.** একই origin-এ CORS সমস্যা অনেক কমে।
88. **MCQ 88 — উত্তর: B.** ফাঁস হওয়া secret বদলাতেই হয়; শুধু মুছলে ইতিহাসে থাকে।
89. **MCQ 89 — উত্তর: B.** secret কোড থেকে আলাদা রাখাই নিরাপত্তার ভিত্তি।
90. **MCQ 90 — উত্তর: B.** মাইগ্রেশন ও কোডের সামঞ্জস্য না ভাবলে ডেপ্লয়েই ভাঙে।
91. **MCQ 91 — উত্তর: B.** blue-green দ্রুত ফেরার পথসহ নিরাপদ সুইচ দেয়।
92. **MCQ 92 — উত্তর: B.** rolling-এ দুই সংস্করণ একসঙ্গে চলে, তাই সামঞ্জস্য অপরিহার্য।
93. **MCQ 93 — উত্তর: B.** smoke test ছাড়া ভাঙা ডেপ্লয় ব্যবহারকারীরাই প্রথম টের পায়।
94. **MCQ 94 — উত্তর: B.** ফেরার পথ ছাড়া ডেপ্লয় ঝুঁকি নেওয়া।
95. **MCQ 95 — উত্তর: B.** ইনসিডেন্টে আগে প্রভাব কমানো, দোষ পরে।
96. **MCQ 96 — উত্তর: B.** blameless সংস্কৃতি ছাড়া আসল কারণ চাপা পড়ে।
97. **MCQ 97 — উত্তর: B.** সময়রেখা ছাড়া মূল কারণ অনুমাননির্ভর হয়।
98. **MCQ 98 — উত্তর: B.** একটি মেট্রিক নয়, সামগ্রিক চিত্রই বাধা দেখায়।
99. **MCQ 99 — উত্তর: B.** অপ্রয়োজনীয় alert আসল বিপদকে চাপা দেয়।
100. **MCQ 100 — উত্তর: B.** রক্ষণাবেক্ষণ মানে যাচাইয়ের নিয়মিত চক্র, একটিমাত্র কাজ নয়।

101. **MCQ 101 — উত্তর: B.** বারবার restart মূল কারণ নয়, লক্ষণ ঢাকার উপায়।
102. **MCQ 102 — উত্তর: B.** প্রোডাকশনে স্থিতিশীলতাই প্রথম, পরীক্ষা ল্যাবে।
103. **MCQ 103 — উত্তর: B.** চেকলিস্ট চাপের মধ্যে ভুল কমায়।
104. **MCQ 104 — উত্তর: B.** ping ব্যর্থ মানে অ্যাপের আগেই নেটওয়ার্ক/হোস্ট স্তরে সমস্যা।
105. **MCQ 105 — উত্তর: B.** সাইট চললে সার্ভার চলছে; সমস্যা SSH স্তরে।
106. **MCQ 106 — উত্তর: B.** I/O-র উৎস না জেনে হার্ডওয়্যার বাড়ানো অনুমান।
107. **MCQ 107 — উত্তর: B.** static চললে proxy ঠিক আছে; সমস্যা অ্যাপ স্তরে।
108. **MCQ 108 — উত্তর: B.** নষ্ট ব্যাকআপ প্রয়োজনের সময় ধরা পড়লে অনেক দেরি।
109. **MCQ 109 — উত্তর: B.** অজানা প্রসেস kill করার আগে শনাক্তকরণ জরুরি।
110. **MCQ 110 — উত্তর: B.** ব্যাকআপ-পরিবর্তন-যাচাই চক্রই প্রোডাকশনের নিরাপত্তা।
111. **MCQ 111 — উত্তর: B.** status, লগ ও আসল অনুরোধ — তিনটি মিলেই নিশ্চয়তা।
112. **MCQ 112 — উত্তর: B.** রানবুক ছাড়া জ্ঞান একজনের মাথায় আটকে থাকে।
113. **MCQ 113 — উত্তর: B.** অনথিভুক্ত ও অপরীক্ষিত ব্যবস্থাই দীর্ঘমেয়াদে ভাঙে।
114. **MCQ 114 — উত্তর: B.** অ্যাক্সেস দেওয়ার আগে সীমা ও জবাবদিহি ঠিক করাই সঠিক।
115. **MCQ 115 — উত্তর: B.** দায়িত্ব ও সময়সীমা ছাড়া প্রতিরোধ শুধু ইচ্ছা।
116. **MCQ 116 — উত্তর: B.** সার্ভার হাতে-কলমে ভেঙে-গড়েই শেখা যায়।
117. **MCQ 117 — উত্তর: B.** সতর্কতা, যাচাই ও শেখাই প্রোডাকশন দক্ষতা, মুখস্থ নয়।
118. **MCQ 118 — উত্তর: B.** অজানা প্যাকেজের প্রভাব না জেনে ইনস্টল করা ঝুঁকিপূর্ণ।
119. **MCQ 119 — উত্তর: B.** পুনরাবৃত্ত ত্রুটি মানে মূল কারণ এখনো আছে।
120. **MCQ 120 — উত্তর: B.** ব্যাকআপ, পরিবর্তন, যাচাই ও নথিভুক্তকরণ — এই ক্রমই নিরাপদ।

সংক্ষিপ্ত উত্তরের মডেল উত্তর (SA ১–৩০)

1. **SA 1.** প্রোডাকশন সার্ভার হলো আসল ব্যবহারকারীদের সেবা দেওয়া সার্ভার। পার্থক্য: (১) এটি ২৪/৭ চলতে হয় ও ব্যর্থতার সরাসরি প্রভাব পড়ে; (২) পরিবর্তনে ব্যাকআপ, যাচাই ও রোলব্যাক পরিকল্পনা বাধ্যতামূলক।
2. **SA 2.** LTS-এর কারণ: (১) দীর্ঘমেয়াদি নিরাপত্তা আপডেট; (২) স্থিতিশীল প্যাকেজ সংস্করণ, ফলে অপ্রত্যাশিত পরিবর্তন কম।
3. **SA 3.** ব্যবহারকারীর private key নিজের কাছে থাকে, public key সার্ভারে থাকে। লগইনের সময় সার্ভার একটি চ্যালেঞ্জ দেয়, private key দিয়ে তার উত্তর মেলে কি না যাচাই হয় — পাসওয়ার্ড পাঠানো হয় না।
4. **SA 4.** ন্যূনতম অধিকার মানে শুধু প্রয়োজনীয় কাজের অনুমতি দেওয়া। উদাহরণ: ডেভেলপারকে শুধু systemctl restart app-এর sudo দেওয়া, সব কমান্ডের নয়।
5. **SA 5.** LVM-এর সুবিধা: (১) ভলিউমের আকার পরে বাড়ানো যায়; (২) snapshot নিয়ে ব্যাকআপ/পরীক্ষা করা যায়।
6. **SA 6.** inode শেষ হলে জায়গা থাকলেও নতুন ফাইল তৈরি হয় না। উদাহরণ: লাখো ছোট ক্যাশ ফাইল তৈরি করা অ্যাপে inode শেষ হয়ে লেখা বন্ধ হয়ে যায়।
7. **SA 7.** load average হলো নির্দিষ্ট সময়ে CPU-র জন্য অপেক্ষমাণ কাজের গড়। 4 CPU-তে load 8 মানে গড়ে ৮টি কাজ অপেক্ষায় — CPU চাপে আছে।
8. **SA 8.** SIGTERM প্রসেসকে পরিষ্কারভাবে থামার সুযোগ দেয়; SIGKILL জোর করে থামায়, সংরক্ষণের সুযোগ দেয় না। তাই প্রথমে SIGTERM, তারপর SIGKILL।

9. **SA 9.** apt update প্যাকেজের তালিকা হালনাগাদ করে; apt upgrade তালিকা অনুযায়ী প্যাকেজ ইনস্টল করে।
10. **SA 10.** systemctl enable boot-এ চালুর ব্যবস্থা করে; systemctl start এখনই সার্ভিস চালায়।
11. **SA 11.** journalctl -u nginx -f nginx সার্ভিসের লগ সরাসরি (follow) দেখায়। সার্ভিসের ত্রুটি সরাসরি দেখতে ব্যবহার করা হয়।
12. **SA 12.** Reverse proxy হলো ক্লায়েন্ট ও অ্যাপের মাঝখানের সার্ভার, যা অনুরোধ গ্রহণ করে অ্যাপে পাঠায়। সুবিধা: TLS, static ফাইল ও লোড সামলানো proxy-র কাজ হয়।
13. **SA 13.** 502-এর কারণ: (১) পেছনের অ্যাপ বন্ধ; (২) অ্যাপ ভুল পোর্টে শুনছে বা সাড়া দিচ্ছে না।
14. **SA 14.** nginx -t কনফিগের ভুল যাচাই করে; nginx -s reload ভুল না থাকলে সংযোগ না ভেঙে নতুন কনফিগ লোড করে।
15. **SA 15.** Let's Encrypt সার্টিফিকেটের মেয়াদ প্রায় ৯০ দিন; নবায়ন ভুলে গেলে ব্রাউজার সাইটকে অবিশ্বস্ত দেখায়, তাই স্বয়ংক্রিয় নবায়ন জরুরি।
16. **SA 16.** NODE_ENV=production-এর প্রভাব: (১) production অপ্টিমাইজেশন চালু হয়; (২) বিস্তারিত ডেভেলপমেন্ট ত্রুটি-বার্তা লুকানো হয়।
17. **SA 17.** PM2-এর সুবিধা: cluster mode ও সহজ প্রসেস ব্যবস্থাপনা। systemd-এর সুবিধা: boot, লগ ও সার্ভারব্যাপী একক সার্ভিস ব্যবস্থা।
18. **SA 18.** প্রতিটি অনুরোধে নতুন সংযোগ খুললে ডেটাবেস চাপে পড়ে; pool নির্দিষ্ট সংখ্যক সংযোগ পুনরায় ব্যবহার করে চাপ নিয়ন্ত্রণ করে।
19. **SA 19.** auth ছাড়া MongoDB উন্মুক্ত থাকলে যে কেউ ডেটা পড়তে, বদলাতে বা মুছতে পারে — সরাসরি ডেটা-চুরির ঝুঁকি।
20. **SA 20.** pg_dump-এর পর restore করে যাচাই করতে হয়; restore না করলে ব্যাকআপ কাজ করবে তার প্রমাণ নেই।
21. **SA 21.** logrotate পুরোনো লগ compress/মুছে লগের আকার নিয়ন্ত্রণ করে। না থাকলে লগ বেড়ে ডিস্ক ভরে যায়।
22. **SA 22.** available হলো সত্যিই ব্যবহারযোগ্য RAM (cache বাদে); used-এ cache ধরা থাকে বলে available-ই দেখতে হয়।
23. **SA 23.** default deny মানে সব বন্ধ, শুধু প্রয়োজনীয় পোর্ট খোলা — আক্রমণের পথ কমে।
24. **SA 24.** fail2ban লগে বারবার ব্যর্থ লগইন দেখে ওই IP-কে সাময়িকভাবে ফায়ারওয়ালে বন্ধ করে দেয়।
25. **SA 25.** 3-2-1: ডেটার ৩ কপি, ২ ভিন্ন মাধ্যমে, ১টি অফ-সাইটে — যাতে এক স্থানের ক্ষতিতে সব না হারায়।
26. **SA 26.** cron-এর PATH ইন্টার‍্যাক্টিভ শেলের মতো নয়, তাই সংক্ষিপ্ত কমান্ড নাও মিলতে পারে। সমাধান: পূর্ণ পাথ ব্যবহার করা।
27. **SA 27.** ধাপ: (১) সংশ্লিষ্ট secret ঘোরানো (rotate); (২) git ইতিহাস থেকে ফাইল সরানো ও .gitignore ঠিক করা। শুধু মুছলেই হবে না।
28. **SA 28.** Blue-green: দুটি অভিন্ন পরিবেশ (blue/green); নতুন সংস্করণ green-এ পরীক্ষা করে ট্রাফিক সুইচ করা হয়। সুবিধা: ব্যর্থ হলে দ্রুত পুরোনোটায় ফেরা যায়।
29. **SA 29.** Blameless পোস্টমর্টেম মানে ব্যক্তিকে দোষ না দিয়ে প্রক্রিয়া ও মূল কারণ খোঁজা, যাতে সত্য তথ্য বের হয়।
30. **SA 30.** ভালো রানবুকে থাকে: (১) আর্কিটেকচার ও অ্যাক্সেস; (২) নিয়মিত কাজের ধাপ; (৩) ইনসিডেন্ট পদ্ধতি, যোগাযোগ ও রোলব্যাক।

পরিস্থিতিভিত্তিক প্রশ্নের মডেল উত্তর (SC ১–২০)

1. **SC 1.** Nginx ও Node সার্ভিসের status, পোর্টে কে শুনছে (ss), লগের শেষ লাইন — এই ক্রমে। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
2. **SC 2.** du দিয়ে বড় ডিরেক্টরি শনাক্ত, লগ rotate/compress, খোলা ফাইল যাচাই; ডেটাবেস/অ্যাপ ডেটা অনুমান করে মোছা নয়। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
3. **SC 3.** প্রসেস শনাক্ত, লগ/মেট্রিক দেখা, infinite loop না ট্রাফিক-চাপ বোঝা, প্রয়োজনে graceful restart ও কারণ নথিভুক্ত করা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
4. **SC 4.** Nginx error লগ, upstream পোর্টে অ্যাপ চলছে কি না, env/পোর্ট বদলেছে কি না, শেষ ডেপ্লয়ের পরিবর্তন। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
5. **SC 5.** key ফাইল, অনুমতি (700/600), authorized_keys, sshd_config, ফায়ারওয়াল — সার্ভারে console/alternate পথে। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
6. **SC 6.** তাৎক্ষণিক: অপ্রয়োজনীয় সংযোগ বন্ধ/অ্যাপ restart; দীর্ঘমেয়াদি: pool সীমা, pooler, সংযোগ লিক ঠিক করা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
7. **SC 7.** certbot লগ দেখা, পোর্ট 80/চ্যালেঞ্জ পথ যাচাই, dry-run, প্রয়োজনে হাতে নবায়ন ও মনিটরিং alert বসানো। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
8. **SC 8.** cron সার্ভিস, crontab সিনট্যাক্স, টাইমজোন, PATH/পূর্ণ পাথ, স্ক্রিপ্টের অনুমতি ও আউটপুট রিডাইরেক্ট। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
9. **SC 9.** সার্ভিস enable করা হয়নি, EnvironmentFile পাথ ভুল, ডেটাবেস তখনো ওঠেনি (After=/Requires)। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
10. **SC 10.** oplog/লগ আকার, অপ্রয়োজনীয় সংগ্রহ, বড় ডকুমেন্ট, index আকার — du ও db.stats দিয়ে। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
11. **SC 11.** লগে ধরন যাচাই (বৈধ বট না আক্রমণ), rate limit, সাময়িক ব্লক, upstream চাপ পরীক্ষা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
12. **SC 12.** journalctl/status, nginx -t, কনফিগ পরিবর্তন/ব্যাকআপ থেকে তুলনা, প্রয়োজনে আগের সংস্করণে ফেরা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
13. **SC 13.** মেয়াদোত্তীর্ণ/ভুল ডোমেইনের সার্টিফিকেট, mixed content, redirect লুপ। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
14. **SC 14.** হোস্টিং console/VNC, বিকল্প পোর্ট/ব্যবহারকারী, নিয়ম ফেরানো — আগে থেকে ফেরার পথ রাখা উচিত ছিল। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
15. **SC 15.** কোন প্রসেস RAM নিচ্ছে তা শনাক্ত, memory leak না ট্রাফিক-চাপ বোঝা, restart নয় — কারণ ঠিক করা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
16. **SC 16.** ক্যাশ (browser/CDN/Nginx), rolling-এ দুই সংস্করণ, load balancer-এ পুরোনো নোড। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
17. **SC 17.** inode শেষ, বা মুছে ফেলা খোলা ফাইল, বা /tmp-এর আলাদা সীমা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।

18. **SC 18.** অন্য কপি (অফ-সাইট/পুরোনো) খোঁজা, ঘটনা নথিভুক্ত করা, ভবিষ্যতে checksum ও restore পরীক্ষা বাধ্যতামূলক করা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
19. **SC 19.** একজন যোগাযোগকারী ঠিক করা, নির্দিষ্ট সময়ে সংক্ষিপ্ত আপডেট, কাজের ধারা না ভাঙা। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।
20. **SC 20.** প্রসেস/সংযোগ শনাক্ত, নেটওয়ার্ক আলাদা করা, প্রমাণ সংরক্ষণ, অনুমান করে মুছে ফেলা নয়। যাচাই: প্রতিটি ধাপের পর status/লগ/আসল অনুরোধ দিয়ে নিশ্চিত করুন এবং যা করলেন তা নথিভুক্ত করুন।

কেস স্টাডির মূল্যায়ন-নোট (CS ১–১০)

1. **CS 1.** পূর্ণ নম্বর: মূল কারণ (logrotate কনফিগ) ধরা, মুছে ফেলা শুধু সাময়িক — স্বীকার, alert + retention নীতি প্রস্তাব। আংশিক: শুধু 'লগ মুছুন'।
2. **CS 2.** পূর্ণ নম্বর: env/পোর্ট অসামঞ্জস্য ধরা, ডেপ্লয় চেকলিস্টে পোর্ট যাচাই ও smoke test, লিখিত রোলব্যাক ধাপ। আংশিক: শুধু 'আগের সংস্করণে ফিরুন'।
3. **CS 3.** পূর্ণ নম্বর: তাৎক্ষণিক rotate, ইতিহাস পরিষ্কার, .gitignore + pre-commit hook, ডেটাবেস লগ পর্যালোচনা। আংশিক: শুধু ফাইল মোছা।
4. **CS 4.** পূর্ণ নম্বর: key পরীক্ষা ছাড়া পাসওয়ার্ড বন্ধ করা ভুল, console দিয়ে ফেরা, ভবিষ্যতে 'পরীক্ষা→তারপর বন্ধ' নীতি। আংশিক: শুধু 'console ব্যবহার করুন'।
5. **CS 5.** পূর্ণ নম্বর: restore পরীক্ষা না করাই মূল ব্যর্থতা; নিয়মিত restore মহড়া, checksum, অফ-সাইট কপি প্রস্তাব। আংশিক: শুধু 'ব্যাকআপ নিন'।
6. **CS 6.** পূর্ণ নম্বর: reboot লক্ষণ ঢাকে; ব্যাচ জবের সময়/সীমা বদল, nice, লগ বিশ্লেষণ। আংশিক: শুধু 'জব বন্ধ করুন'।
7. **CS 7.** পূর্ণ নম্বর: স্বয়ংক্রিয় নবায়ন ব্যর্থ হতে পারে (পোর্ট/চ্যালেঞ্জ); মেয়াদ-মনিটরিং alert ও দায়িত্বপ্রাপ্ত ব্যক্তি। আংশিক: শুধু 'হাতে নবায়ন করুন'।
8. **CS 8.** পূর্ণ নম্বর: সংযোগ লিক/pool-এর অভাব মূল কারণ; pool সীমা, pooler, কোড পর্যালোচনা। আংশিক: শুধু 'সংযোগ বাড়ান'।
9. **CS 9.** পূর্ণ নম্বর: স্তর ধরে পরীক্ষা না করাই ভুল; নেটওয়ার্ক→ফায়ারওয়াল→Nginx→অ্যাপ ক্রম ব্যাখ্যা। আংশিক: শুধু 'ফায়ারওয়াল দেখুন'।
10. **CS 10.** পূর্ণ নম্বর: একজন সমন্বয়কারী, নির্দিষ্ট আপডেট ছক, সময়রেখা; পোস্টমর্টেমে প্রক্রিয়াগত প্রতিরোধ। আংশিক: শুধু 'যোগাযোগ ভালো করুন'।

চূড়ান্ত পরীক্ষার মূল্যায়ন-নোট

তত্ত্ব: MCQ-তে সঠিক উত্তরে ১ নম্বর, ভুলে শূন্য (ঋণাত্মক নয়)। সংক্ষিপ্ত উত্তরে মূল ধারণা + উদাহরণ থাকলে পূর্ণ নম্বর। ব্যবহারিক: প্রতিটি কাজে দাবির পাশাপাশি প্রমাণ (status/লগ/curl আউটপুট) দেখাতে হবে; প্রমাণ ছাড়া দাবিতে নম্বর নেই। পাসের শর্ত: মোট ৬০+, তত্ত্বে ২০+, ব্যবহারিকে ৩০+।